Marine Ecosystem Services Processing

Fish provisioning: annotated R processing routine

Table of contents

Introduction 2
- Preliminaries . . . 3

Reference tables 4
- Sub-region harmonisation . . . 4
- Sea region assignment . . . 5
- Species typology . . . 6

Helper functions 7

Economic component 8
- Loading the STECF economic table . . . 8
- Net economic result by fleet . . . 8

Biophysical component 9
- Loading and cleaning landings . . . 9
- Harmonising regions and attaching the typology . . . 9

Extending the series back to 2002 13
- Loading the workbook . . . 13
- Net economic result by fleet . . . 14
- Landings . . . 14
- Checks on the 2011 data call . . . 16
- Overlap with STECF in 2008 and 2009 . . . 18
- Joining the two sources . . . 20

Resource rent allocation 20
- Matching landings to fleet economics . . . 20
- Distributing the net result . . . 21
- Observed prices . . . 21
- Gap set . . . 22
- Filling the gaps . . . 22

Final dataset 23

Accounting tables 27

Outputs 33
Joining results to the marine regions 41
Downloads 45
Decisions from JRC (September 2026) 46
References 46

Introduction

This page documents the R routine that turns the Scientific, Technical and Economic Committee for Fisheries (STECF) extraction (Scientific, Technical and Economic Committee for Fisheries, 2024) into the fish provisioning accounts. Fish provisioning is the ecosystem contribution to the growth of aquatic biomass captured in uncultivated production contexts (Haines-Young & Potschin, 2018), and it is valued here with the resource rent approach of the SEEA Central Framework (United Nations et al., 2014). The accompanying methods report states the equations; this page restates the earlier working script in a compact, reproducible form so that the team can inspect each step, reproduce it from the repository alone, and reuse the parts that generalise.

The routine reads two committed RDS files produced from the STECF fleet-segment workbook: `FS_data_2008-2024.RDS` for income and expenditure, and `FS_landings_FAO_2008-2024.RDS` for landings in kilograms and euro. Kilograms exist only in the landings file; income and expenditure exist only in `FS data`. The conversion is documented on the STECF Fleet Segment Data page.

Years 2002 to 2007 come from a second, older source: the 2011 Data Collection Framework (DCF) data call, supplied by JRC in October 2026 as `data/dcf/DCF_2011_fleet_economic_and_landings_2002-2010.xlsx`. That workbook has a different fleet definition and a different economic variable, so it is prepared in its own section, Extending the series back to 2002, and joins the pipeline just before the allocation.

The routine has three components, and they run in this order because the second depends on the first:

1. the **economic component**, which reduces the STECF economic variables to a single net economic result per fleet segment and year (Eq. 3 of the methods report);
2. the **biophysical component**, which cleans the landings table, harmonises fishing sub-regions, assigns each sub-region to a European sea region, and attaches the species typology;
3. the **resource rent allocation**, which distributes each fleet's net economic result across the sub-regions and species it landed, then fills the gaps left by fleets without usable economic data.

> **!** 2024 is excluded from the valuation
>
> The STECF files run to 2024, but no Member State has reported a usable expenditure series for that year. The latest fully valuable year is 2023. The two year filters below therefore keep `year <= reference_year` and drop 2024. Landings for 2024 can be shown in physical terms with a provisional caveat; they cannot carry a resource rent.

> **i** Monetary values are 2010 euro
>
> Allocation is done in nominal euro. A single step at the end converts resource rent to constant 2010 prices with Eurostat HICP for EU-27 (Eurostat, 2026). The base year is `base_year` in the parameters chunk (set to 2010 after consultation with JRC); changing it does not touch the allocation. Tables

report both nominal and 2010 euro. Physical landings are not deflated.

> **Paths are relative to the project root**
>
> `_quarto.yml` sets `execute-dir: project`, so every path below resolves from the repository root regardless of where the `.qmd` file sits. There is no need for `here::here()` or `setwd()`.

Preliminaries

We work in the tidyverse throughout. `readxl` reads the lookup workbooks, `sf` joins the results to the marine region geometry, `writexl` writes the flat panel workbook, `openxlsx2` writes the formatted results workbook in the Outputs section, and `gt` formats the inspection tables. `writexl` cannot set number formats, column widths or borders, so it is kept only for the unformatted panel.

```
library(tidyverse)
library(readxl)
library(sf)
library(writexl)
library(openxlsx2)
library(gt)

# In the PDF, every gt table uses a 7px font so wide tables fit the page
if (knitr::is_latex_output()) {
  gt <- function(...) {
    gt::gt(...) |>
      tab_options(table.font.size = px(7))
  }
}

sf_use_s2(FALSE)
```

Every input and output path is declared once, at the top, so that a reader can see the full data footprint of the routine without scrolling. `reference_year` is the latest fully valuable year. The two year filters use `year <= reference_year` rather than `year == reference_year`; `year` is already part of every grouping key and every gap-filling join, so the result is a 2002 to 2023 panel with no cross-year contamination. `dcf_years` are the years taken from the 2011 DCF data call; every later year comes from STECF.

```
reference_year <- 2023
base_year <- 2010
dcf_years <- 2002:2007

# Inputs
stecf_economic_file <- "data/stecf/FS_data_2008-2024.RDS"
stecf_landings_file <- "data/stecf/FS_landings_FAO_2008-2024.RDS"
dcf_file <- "data/dcf/DCF_2011_fleet_economic_and_landings_2002-2010.xlsx"
country_codes_file <- "data/stecf/country_codes.csv"
species_lookup_file <- "data/lookup/3.classification_vs_OFFICIAL.xlsx"
gsa_lookup_file <- "data/lookup/Link_GSA_Marine regions_Mediterranean Sea.xlsx"
```

```
overuse_lookup_file <- "data/lookup/Overuse_Total.xlsx"
hicp_file <- "data/lookup/hicp_eu27.csv"
gis_file <- "data/gisdata/marine_regions.gpkg"

# Outputs
results_dir <- "data/outputs/results"
gis_dir <- "data/outputs/gis"
dir.create(results_dir, showWarnings = FALSE, recursive = TRUE)
dir.create(gis_dir, showWarnings = FALSE, recursive = TRUE)

country_names <- readr::read_csv(country_codes_file, show_col_types = FALSE) |>
  transmute(country_code = `alpha-3`, country = name) |>
  mutate(
    country = recode(country, `Netherlands, Kingdom of the` = "Netherlands")
  )
```

Reference tables

The classification decisions of the assessment were previously buried inside long `case_when()` blocks or made by hand in Excel. They are collected here instead, as tables wherever a table will do. The team can then review and amend them without reading the pipeline, and they are easy to export for the annex of the report.

Sub-region harmonisation

STECF reports fishing areas at a finer resolution than the accounts require, and it mixes two naming systems: FAO fishing areas such as `37.1.1` and General Fisheries Commission for the Mediterranean sub-areas such as `sa21`. The table below collapses the finer codes onto the code actually used in the accounts. It reproduces the recodes of the original script, plus three collapses Domenico asked for: `sa11.1` and `sa11.2` onto `sa11` (Mediterranean), the Baltic subdivisions `27.3.d.28.1` and `27.3.d.28.2` onto `27.3.d.28`, and the parent ICES area `27.8` onto `27.8.a`.

```
sub_reg_recodes <- tribble(
  ~from,       ~to,
  "37.1.1",    "sa5",
  "sa21.1",    "sa21",
  "sa21.2",    "sa21",
  "sa21.3",    "sa21",
  "37.2.2",    "sa21",
  "37.1.3",    "sa21",
  "37.3.1",    "sa22",
  "27.2.a.1",  "27.2.a",
  "27.2.a.2",  "27.2.a",
  "27.2.b.1",  "27.2.b",
  "27.2.b.2",  "27.2.b",
  "27.3.a.20", "27.3.a",
  "27.3.a.21", "27.3.a",
  "27.3.c.22", "27.3.c",
```

```
  "27.3.b.23", "27.3.b",
  "27.5.a.1",  "27.5.a",
  "27.5.a.2",  "27.5.a",
  "27.5.b.1",    "27.5.b",
  "27.5.b.2",    "27.5.b",
  "27.14.b.1",   "27.14.b",
  "27.14.b.2",   "27.14.b",
  "sa11.1",      "sa11",
  "sa11.2",      "sa11",
  "27.3.d.28.1", "27.3.d.28",
  "27.3.d.28.2", "27.3.d.28",
  "27.8",        "27.8.a"
)
```

The correspondence between GFCM sub-areas and FAO Mediterranean divisions is held separately in `data/lookup`. The pipeline does not consume it; it is the reference that justifies the Mediterranean rows of the recode table, and it is the authority against which the sea region rules below can be audited.

```
gsa_to_fao <- read_excel(gsa_lookup_file, sheet = "Regions division") |>
  rename(sub_reg = `Row Labels`, fao_division = Sea_Region)
```

Sea region assignment

Each harmonised sub-region belongs to one of the four pan-European marine regions of the Marine Strategy Framework Directive, or to the rest of the world when a European fleet lands a catch taken elsewhere. The rules are a direct transcription of the original `case_when()`; wrapping them in a function means the same classification can be reused by the overuse and blue carbon routines without copying the list of codes again.

```
neao_sub_regions <- c(
  "27.1.a",  "27.1.b",  "27.10.a", "27.10.b", "27.2.a",  "27.2.b",
  "27.3.a",  "27.3.c",  "27.3.b",  "27.4.a",  "27.4.b",  "27.4.c",
  "27.5.a",  "27.5.b",  "27.6.a",  "27.6.b",  "27.7.a",  "27.7.b",
  "27.7.c",  "27.7.d",  "27.7.e",  "27.7.f",  "27.7.g",  "27.7.h",
  "27.7.j",  "27.7.k",  "27.8.a",  "27.8.b",  "27.8.c",  "27.8.d",
  "27.8.e",  "27.9.a",  "27.9.b",  "27.12.a", "27.12.b", "27.12.c",
  "27.14.a", "27.14.b"
)

assign_sea_region <- function(sub_reg_clean) {
  case_when(
    sub_reg_clean == "0" ~ "unknown",
    sub_reg_clean %in% c("sa28", "sa29", "sa30") ~ "Black Sea",
    sub_reg_clean %in% paste0("sa", 1:27) ~ "Mediterranean Sea",
    str_detect(sub_reg_clean, fixed("27.3.d")) ~ "Baltic Sea",
    str_detect(sub_reg_clean, fixed("27.8")) |
      sub_reg_clean %in% neao_sub_regions ~ "NEAO",
    .default = "ROW"
  )
}
```

species_code	species_name	Type	fish	shellfish	Other
CAS	Spotted wolffish	demersal	1	0	0
CAS	Spotted wolffish	demersal	1	0	0
RKQ	Noah's ark	benthic	0	1	0
RKQ	Noah's ark	benthic	0	1	0
SCY	Lesser slipper lobster	benthic	0	1	0
SCY	Lesser slipper lobster	benthic	0	1	0
SHR	Sharpsnout seabream	benthopelagic	1	0	0
SHR	Sharpsnout seabream	benthopelagic	1	0	0
SJA	Great Mediterranean scallop	benthic	0	1	0
SJA	Great Mediterranean scallop	benthic	0	1	0
SRK	Painted comber	demersal	1	0	0
SRK	Painted comber	demersal	1	0	0
SSB	Sand steenbras	demersal	1	0	0
SSB	Sand steenbras	demersal	1	0	0
YRS	European barracuda	pelagic	1	0	0
YRS	European barracuda	pelagic	1	0	0

Species typology

STECF identifies species by their FAO ASFIS code but says nothing about what kind of animal it is or where in the water column it lives. The methods report fills that gap from FishBase and SeaLifeBase (Froese & Pauly, 2024; Palomares & Pauly, 2024), and the result is the classification workbook already in `data/lookup`. We derive two attributes from it: `species_group`, which separates fish from shellfish and from species whose code could not be matched with confidence, and `fish_type`, the aggregated water column typology of Table 1 of the methods report. Those names match the nested columns of methods Tables 4 to 13. The workbook column `Type` holds the water-column codes (benthic, benthopelagic, demersal, pelagic); the fish / shellfish / other split comes from the binary flags. There is no `Type_1` column in the lookup.

```
species_types_raw <- read_excel(species_lookup_file, sheet = "Classification")

species_types <- species_types_raw |>
  transmute(
    species_code,
    species_name_lookup = species_name,
    fish_type = Type,
    species_group = case_when(
      fish == 1 ~ "fish",
      shellfish == 1 ~ "shellfish",
      .default = "other"
    )
  ) |>
  distinct(species_code, .keep_all = TRUE)
```

> **i** Duplicate species codes are identical pairs
>
> The original script joined landings to this classification by hand in Excel because the join in R produced double counting. Eight species codes appear twice in the `Classification` sheet (CAS, RKQ, SCY, SHR, SJA, SRK, SSB, YRS). Each pair is identical on name, `Type`, and the fish / shellfish flags, so `distinct(species_code, .keep_all = TRUE)` is a safe delete. Domenico confirmed that dropping the duplicates is fine.

```
species_types_raw |>
  filter(n() > 1, .by = species_code) |>
  arrange(species_code) |>
  select(species_code, species_name, Type, fish, shellfish, Other) |>
  gt()
```

Helper functions

These operations recur often enough to be worth naming. Everything else stays as an explicit pipeline, because a named function that is called once hides more than it saves.

Both STECF sheets carry the same block of administrative columns that the assessment never uses. `any_of()` rather than `all_of()` means the function works on both sheets even though they do not carry exactly the same set.

```
drop_stecf_admin_columns <- function(data) {
  data |>
    select(-any_of(c(
      "upload_date", "geo_indicator", "cluster_name", "fromtable",
      "framework", "template_name", "gear", "fishery", "activity"
    )))
}
```

Harmonising sub-regions is two steps: strip the stray whitespace that STECF sometimes carries, then apply the recode table. We strip first so that a code entered as `27.2.a.1` still matches. The raw `sub_reg` is kept alongside `sub_reg_clean` for traceability.

```
harmonise_sub_reg <- function(data, recodes = sub_reg_recodes) {
  data |>
    mutate(sub_reg_clean = str_remove_all(sub_reg, "\\s")) |>
    left_join(
      recodes,
      by = join_by(sub_reg_clean == from),
      relationship = "many-to-one"
    ) |>
    mutate(sub_reg_clean = coalesce(to, sub_reg_clean)) |>
    select(-to)
}
```

The gap filling described in Box 2 of the methods report is the same operation applied three times at successively coarser levels of aggregation. Expressing it once as a function that takes the grouping columns as an argument removes about sixty lines of copied code and guarantees that the three passes stay consistent with each other.

```
# Mean observed price of a species over an arbitrary set of grouping columns.
average_price <- function(reference, by) {
  reference |>
    summarise(price = mean(price, na.rm = TRUE), .by = all_of(by))
}

# Assign that mean price to records that have landings but no usable rent.
# Records that still find no reference price keep a price of zero and are
# passed on to the next, coarser pass.
fill_price_gaps <- function(gaps, reference, by, basis) {
  gaps |>
    select(-any_of(c("price", "sum_revenue", "price_basis"))) |>
```

```
    left_join(
      average_price(reference, by),
      by = by,
      relationship = "many-to-one"
    ) |>
    mutate(
      price = replace_na(price, 0),
      price_basis = if_else(price > 0, basis, NA_character_),
      sum_revenue = sum_landed * price
    )
}
```

Economic component

Loading the STECF economic table

The economic file reports one row per fleet segment, year and economic variable. Inactive fleets are removed first because they land nothing and would otherwise distort the fleet averages. The year filter keeps the panel through `reference_year` and drops 2024, which cannot be valued.

```
stecf_economic <- readRDS(stecf_economic_file) |>
  filter(fishing_tech != "INACTIVE", year <= reference_year) |>
  drop_stecf_admin_columns()
```

Net economic result by fleet

The resource rent approach values the service as the economic surplus of the fleet, that is total income less total costs (Eq. 3 of the methods report). STECF splits its economic variables into groups; we keep only `Expenditure` and `Income`, sign the expenditure negatively so that both can be summed in a single column, and aggregate to one net result per fleet segment and year.

Missing values become zero and zero rows are then dropped. That looks circular, and it is deliberate: STECF uses a blank to mean "not reported" and the assessment treats a non-reported item as absent rather than as a true zero, so both end up excluded from the sum.

```
net_economic_by_fleet <- function(data) {
  data |>
    filter(variable_group %in% c("Expenditure", "Income")) |>
    mutate(value = replace_na(value, 0)) |>
    filter(value != 0) |>
    mutate(money = if_else(variable_group == "Expenditure", -value, value)) |>
    summarise(
      total_expenditure = sum(money[variable_group == "Expenditure"]),
      total_income = sum(money[variable_group == "Income"]),
      total_net = total_income + total_expenditure,
      .by = c(year, fs_name)
    )
}
```

```r
fleet_net <- net_economic_by_fleet(stecf_economic)
```

An ecosystem service cannot have a negative flow, because that would describe a disservice. Box 2 of the methods report therefore splits the fleets in two: those with a positive net result carry the valuation, and those with a negative or zero result are treated as missing and recovered later from average prices.

```r
fleet_net_positive <- fleet_net |>
  filter(total_net > 0)
```

Biophysical component

Loading and cleaning landings

The landings file reports quantities in several units. Only kilograms are relevant here, and rows with a zero quantity carry no information. We rename `value` to `landed_kg` immediately, because three different quantities are called `value` at various points of the original script and the name stops carrying meaning. The same `year <= reference_year` bound used on the economic file keeps the two sides of the join on the same years.

```r
landings_raw <- readRDS(stecf_landings_file) |>
  filter(year <= reference_year) |>
  drop_stecf_admin_columns() |>
  filter(unit == "kg", value != 0) |>
  rename(landed_kg = value)
```

Harmonising regions and attaching the typology

Sub-region harmonisation, sea region assignment and the species join all happen in one pipeline. The species join is the step that was previously done by hand in Excel; `relationship = "many-to-one"` makes `dplyr` fail loudly if the lookup ever regains duplicate codes, which is the guard the manual workflow lacked.

```r
landings <- landings_raw |>
  harmonise_sub_reg() |>
  mutate(Sea_reg = assign_sea_region(sub_reg_clean)) |>
  left_join(
    species_types,
    by = join_by(species_code),
    relationship = "many-to-one"
  )
```

Three checks are worth running every time the extraction is refreshed. The first lists species codes present in the landings but absent from the classification workbook; those rows would otherwise reach the accounting tables with a missing fish type. The second compares the species name STECF reports against the name the classification workbook holds for the same code, which is how a mis-assigned code shows itself. The third lists sub-regions that fell through to `ROW`, the catch-all category and therefore the place where a new or misspelled STECF code will hide.

```r
# Too long for the PDF page, so shown in HTML only
if (!knitr::is_latex_output()) {
  landings |>
    filter(year == reference_year, is.na(fish_type)) |>
    distinct(species_code, species_name) |>
    arrange(species_code) |>
    gt()
}

# Too long for the PDF page, so shown in HTML only
if (!knitr::is_latex_output()) {
  landings |>
    filter(year == reference_year, species_name != species_name_lookup) |>
    distinct(species_code, species_name, species_name_lookup) |>
    arrange(species_code) |>
    gt()
}

landings |>
  filter(year == reference_year, Sea_reg == "ROW") |>
  summarise(landed_kg = sum(landed_kg), .by = c(sub_reg, sub_reg_clean)) |>
  arrange(desc(landed_kg)) |>
  gt()
```

A fourth check needs no STECF data at all, so it is the one to look at first. Every sub-area listed in the GFCM correspondence workbook is Mediterranean by definition; any of them that the recode and classification rules fail to place in the Mediterranean is a gap in those rules.

```r
# Expected to be empty, and a header-only table looks broken in the PDF
if (!knitr::is_latex_output()) {
  gsa_to_fao |>
    harmonise_sub_reg() |>
    mutate(assigned = assign_sea_region(sub_reg_clean)) |>
    filter(assigned != "Mediterranean Sea") |>
    gt()
}
```

`sa11.1` and `sa11.2` are recoded to `sa11`

`assign_sea_region()` matches Mediterranean sub-areas against the literal list `sa1` to `sa27`. The decimal codes `sa11.1` and `sa11.2` would otherwise fall through to `ROW`. They are GFCM Mediterranean areas, so the recode table now maps both onto `sa11`. That check should return no rows. The move is a few tens of tonnes in 2023; Mediterranean and ROW totals therefore differ slightly from the shapefile `Tonnes_23` attributes, which were built before this recode.

The 2023 landings, once harmonised and assigned to a sea region, must still match the shapefile `Tonnes_23` attributes for NEAO, the Baltic and the Black Sea. Mediterranean and ROW shift by the tonnage of `sa11.1` and `sa11.2`, which now sit in the Mediterranean.

sub_reg	sub_reg_clean	landed_kg
51.5	51.5	107799921.78
87.2.6	87.2.6	72878985.68
41.3.1	41.3.1	68991515.07
51	51	60645179.29
41.3.2	41.3.2	55072835.28
77	77	37679833.10
34	34	30078770.00
34.3.1	34.3.1	22857107.48
51.4	51.4	17449242.00
27.7.k.2	27.7.k.2	16182425.00
87.1.4	87.1.4	15885831.16
21.3.m	21.3.m	13742570.69
34.1.2	34.1.2	12415135.07
34.3.1.1	34.3.1.1	11448153.00
51.3	51.3	11406373.68
34.4.1	34.4.1	9701985.93
34.1.3	34.1.3	9699253.34
34.3.3	34.3.3	9052720.44
21.3.o	21.3.o	8447245.32
81	81	8187249.76
51.7	51.7	6664721.33
34.3.6	34.3.6	6424575.44
34.3.2	34.3.2	5685130.78
34.4.2	34.4.2	5515978.47
34.2.0	34.2.0	5337951.36
51.6	51.6	5331179.65
47.a.0	47.a.0	5315829.32
21.3.l	21.3.l	5078898.89
31	31	5078224.10
27.7.k.1	27.7.k.1	5067462.00
21.3.n	21.3.n	4907773.09
41.2.4	41.2.4	4797131.23
34.1.3.2	34.1.3.2	4611374.00
47.a.1	47.a.1	4247038.66
51.8	51.8	4000919.28
34.1.3.1	34.1.3.1	3741087.00
47.1.2	47.1.2	2689829.47
47	47	2504974.00
27.7.c.2	27.7.c.2	2443878.00
21.1.a	21.1.a	2281817.41
41.1.4	41.1.4	1911737.10
41.3.3	41.3.3	1766038.68
47.1.1	47.1.1	1695345.66
47.b.0	47.b.0	1675242.93
47.b.1	47.b.1	1475193.31
21.1.c	21.1.c	1427521.31
27.7.j.2	27.7.j.2	1342228.00
47.c.1	47.c.1	1075290.73
21.6.h	21.6.h	1071774.59
41.1.1	41.1.1	1045949.50
34.3.4	34.3.4	994802.58
47.c.0	47.c.0	920271.37
34.2	34.2	840868.00
34.1.1	34.1.1	632314.98
21.1.b	21.1.b	488538.06
41.2.2	41.2.2	467533.67
21.1.d	21.1.d	420011.91
87.1.1	87.1.1	321643.00
47.1.3	47.1.3	319681.73
21.4.v	21.4.v	315330.78
41.2.1	41.2.1	311383.70
48.6	48.6	210610.79
41	41	158615.00
71	71	143538.31
47.d.1	47.d.1	113679.50
34.3.5	34.3.5	101787.88
21.6.g	21.6.g	86835.03
27.6.b.2	27.6.b.2	81015.00
27.5.b.1.b	27.5.b.1.b	67230.00
41.2.3	41.2.3	62074.00
88.2	88.2	55692.01
57.3	57.3	48204.66
57.4	57.4	42029.92
88.1	88.1	36833.27
57.1	57.1	23993.62
57.2	57.2	8540.99
21.6.f	21.6.f	2812.19

```r
anchor_2023 <- tribble(
  ~Sea_reg,             ~anchor_tonnes,
  "NEAO",                     1925814,
  "Baltic Sea",                436672,
  "Mediterranean Sea",         308722,
  "Black Sea",                   9961,
  "ROW",                       713131
)

sa11_tonnes <- landings_raw |>
  filter(
    year == reference_year,
    str_remove_all(sub_reg, "\\s") %in% c("sa11.1", "sa11.2")
  ) |>
  summarise(sa11_tonnes = sum(landed_kg) / 1000) |>
  pull(sa11_tonnes)

anchor_expected <- anchor_2023 |>
  mutate(
    expected_tonnes = case_when(
      Sea_reg == "Mediterranean Sea" ~ anchor_tonnes + sa11_tonnes,
      Sea_reg == "ROW" ~ anchor_tonnes - sa11_tonnes,
      .default = anchor_tonnes
    )
  )

anchor_check <- landings |>
  filter(year == reference_year) |>
  summarise(landed_tonnes = sum(landed_kg) / 1000, .by = Sea_reg) |>
  inner_join(anchor_expected, by = "Sea_reg") |>
  mutate(
    difference_tonnes = landed_tonnes - expected_tonnes,
    match = abs(difference_tonnes) < 0.6
  ) |>
  arrange(Sea_reg)

anchor_check |>
  select(Sea_reg, landed_tonnes, expected_tonnes, difference_tonnes, match) |>
  gt() |>
  fmt_number(c(landed_tonnes, expected_tonnes, difference_tonnes), decimals = 1) |>
  cols_label(
    landed_tonnes = "Landings (tonnes)",
    expected_tonnes = "Expected (tonnes)",
    difference_tonnes = "Difference"
  ) |>
  tab_source_note(
    paste0(
      "Published shapefile Tonnes_23 is the starting point. sa11.1 + sa11.2 = ",
      round(sa11_tonnes, 1),
      " t moved from ROW to the Mediterranean."
```

Sea_reg	Landings (tonnes)	Expected (tonnes)	Difference	match
Baltic Sea	436,671.9	436,672.0	-0.1	TRUE
Black Sea	9,960.6	9,961.0	-0.4	TRUE
Mediterranean Sea	308,742.1	308,742.4	-0.2	TRUE
NEAO	1,925,813.5	1,925,814.0	-0.5	TRUE
ROW	713,110.3	713,110.6	-0.3	TRUE

Published shapefile Tonnes_23 is the starting point. sa11.1 + sa11.2 = 20.4 t moved from ROW to the Mediterranean.

```
    )
  )
```

```
stopifnot(
  "2023 sea-region tonnes must match the published shapefile anchors after the sa11 recode" =
    nrow(anchor_check) == nrow(anchor_expected) && all(anchor_check$match)
)
```

Extending the series back to 2002

The STECF 2025 data call starts in 2008. In October 2026 JRC supplied the landings and economic tables of the 2011 DCF data call, which cover 2002 to 2010, together with a short description by Domenico. The workbook and the description are kept in `data/dcf/`. This section prepares those tables so that they look like the STECF tables to the allocation, which then runs once over the whole 2002 to 2023 panel with no change to its logic.

Only 2002 to 2007 are taken from the 2011 call. From 2008 onward the STECF 2025 call stays the source, for three reasons. It is a later revision of the same years. Its coverage is wider, as the overlap check below shows. Keeping it means the published 2008 to 2023 results do not move. The 2011 workbook does contain 2010, but its economic sheet holds almost nothing for that year and its own notes flag 2010 as provisional.

! What differs in the 2011 data call

- **No fleet segment code.** Following Domenico's description, a fleet is the landing country plus the vessel length class, for example `POL VL1224`. This is coarser than the STECF `fs_name`, which also carries gear and supra-region.
- **Operating cash flow is the net economic result.** The 2011 call has no separate income and expenditure items before 2008, so Eq. 3 is replaced by the reported operating cash flow (OCF), as Domenico instructed. OCF does not deduct consumption of fixed capital or unpaid labour, which the STECF `Expenditure` group does. Rent for 2002 to 2007 is therefore higher than a like-for-like measure would give; the overlap check below sizes the gap.
- **Coarser fishing areas.** Much of the catch is reported at FAO area level (`27.7`, `27.4`, `27.3`) rather than at division level, and the Mediterranean is reported in FAO divisions (`37.1.2`, `37.2.1`) rather than GFCM sub-areas. Supply tables for those years have coarser rows, and the division map covers less of the catch.
- **United Kingdom.** The 2011 call includes the United Kingdom; the STECF 2025 call does not. Its landings and fleets are dropped so that every year covers the same Member States.
- **Small segments.** The workbook notes say that segments of fewer than 10 vessels are excluded.

Loading the workbook

Two of the five sheets are used. The column names are renamed to the STECF names on load, so that the rest of the routine reads the same way for both sources.

```
dcf_landings_raw <- read_excel(dcf_file, sheet = "DCF EU fleet landings 2002-2010") |>
  rename(
    country_code = `MEMBER STATE`, year = YEAR, species_code = SPECIES,
    sub_reg = `SUB REGION`, fishing_tech = `FISHING TECH`,
    vessel_length = `VESSEL LENGTH`, variable = VARIABLE, value = VALUE
  )

dcf_economic_raw <- read_excel(dcf_file, sheet = "EU fleet economic performance") |>
  rename(
    country_code = `MEMBER STATE`, year = YEAR, supra_reg = `SUPRA REGION`,
    fishing_tech = `GEAR TYPE`, vessel_length = `LENGTH CLASS`,
    variable = VARIABLE, value = VALUE
  )

stecf_countries <- sort(unique(landings$country_code))
```

Net economic result by fleet

The economic sheet has one row per country, year, gear and length class. Operating cash flow is summed over gears to give one net result per country and length class, which is the fleet as Domenico defined it. Rows labelled as inactive gear are removed, as on the STECF side. The function takes the years as an argument because the overlap check below calls it again for 2008 and 2009.

```
dcf_fleet_net <- function(data, years) {
  data |>
    filter(
      year %in% years,
      variable == "Operating cash flow (OCF)",
      !str_to_upper(fishing_tech) %in% c("INACTIVE", "NONACTIVE"),
      !is.na(value),
      country_code %in% stecf_countries
    ) |>
    summarise(total_net = sum(value), .by = c(year, country_code, vessel_length)) |>
    mutate(fs_name = paste(country_code, vessel_length))
}

fleet_net_dcf <- dcf_fleet_net(dcf_economic_raw, dcf_years)

fleet_net_dcf_positive <- fleet_net_dcf |>
  filter(total_net > 0) |>
  select(year, fs_name, total_net)
```

Box 2 applies unchanged. A fleet with a negative or zero operating cash flow carries no valuation, and its landings are valued later from average prices.

Landings

Landed weight is `totwghtlandg`, in kilograms. The unit is confirmed against STECF 2025: for Finland and Latvia in 2008 the two calls agree to the kilogram. Rows with a zero or negative weight are removed. The

seven negative weights in the workbook all fall in 2008 and 2009, outside the years used. Sub-region codes are upper case in this workbook (`27.9.A`), so they are lowered before the shared whitespace strip and recode.

Two groups of rows are removed from the recode table for this source. The four rows that turn FAO Mediterranean divisions into GFCM sub-areas are removed because the 2011 call reports the whole Mediterranean in FAO divisions; recoding three divisions to single sub-areas while the others stay as divisions would mix two systems in one year. The `27.8` to `27.8.a` row is removed because in the 2011 call `27.8` is the main reporting code for the Bay of Biscay, about 180,000 tonnes a year; in STECF 2025 it is a residual of a few tonnes.

```
sub_reg_recodes_dcf <- sub_reg_recodes |>
  filter(!str_starts(from, fixed("37.")), from != "27.8")
```

`assign_sea_region()` recognises fine division codes only, so it would send the coarse codes of the 2011 call to ROW. The rule below assigns by FAO major area instead. Area 27 is NEAO, except its Baltic part. Area 37 is the Mediterranean, except subarea `37.4`, which is the Black Sea. `27.3` covers both the Baltic and the Skagerrak and Kattegat; in this workbook it is reported by Poland, Latvia, Estonia and Lithuania, almost all of it sprat, herring and cod, so it is assigned to the Baltic Sea.

```
assign_sea_region_dcf <- function(sub_reg_clean) {
  case_when(
    sub_reg_clean %in% c("0", "0.0") ~ "unknown",
    sub_reg_clean == "27.3" |
      str_starts(sub_reg_clean, fixed("27.3.d")) ~ "Baltic Sea",
    str_detect(sub_reg_clean, "^27(\\.|$)") ~ "NEAO",
    str_starts(sub_reg_clean, fixed("37.4")) ~ "Black Sea",
    str_detect(sub_reg_clean, "^37(\\.|$)") ~ "Mediterranean Sea",
    .default = "ROW"
  )
}
```

The 2011 call reports species by code only. Names are taken from STECF 2025, using the most recent name for each code, then from the classification workbook, and the code itself as a last resort. Three key combinations for Greece in 2003 and 2005 appear twice with different weights; both rows are kept, which sums them, as the allocation does for any two records of the same fleet.

```
clean_dcf_landings <- function(data, years) {
  data |>
    filter(
      year %in% years,
      variable == "totwghtlandg",
      value > 0,
      country_code %in% stecf_countries
    ) |>
    transmute(
      year, country_code, vessel_length, species_code,
      fs_name = paste(country_code, vessel_length),
      sub_reg = str_to_lower(sub_reg),
      landed_kg = value
    ) |>
    harmonise_sub_reg(recodes = sub_reg_recodes_dcf) |>
```

```
    mutate(Sea_reg = assign_sea_region_dcf(sub_reg_clean))
}

stecf_species_names <- landings |>
  filter(!is.na(species_name)) |>
  arrange(desc(year)) |>
  distinct(species_code, species_name)  |>
  distinct(species_code, .keep_all = TRUE)

landings_dcf <- dcf_landings_raw |>
  clean_dcf_landings(dcf_years) |>
  left_join(stecf_species_names, by = "species_code", relationship = "many-to-one") |>
  left_join(species_types, by = join_by(species_code), relationship = "many-to-one") |>
  mutate(species_name = coalesce(species_name, species_name_lookup, species_code))
```

Checks on the 2011 data call

The first check guards the sea rule. Wherever `assign_sea_region()` places a code in a European sea, the rule for the 2011 call must agree; the two may only differ on codes the STECF rule would leave in ROW. The table lists those codes and the tonnage they bring into the European seas.

```
dcf_sea_rule <- landings_dcf |>
  summarise(landed_tonnes = sum(landed_kg) / 1000, .by = c(sub_reg_clean, Sea_reg)) |>
  mutate(stecf_rule = assign_sea_region(sub_reg_clean))

stopifnot(
  "The DCF sea rule may only differ from the STECF rule on codes the STECF rule sends to ROW" =
    with(dcf_sea_rule, all(Sea_reg == stecf_rule | stecf_rule == "ROW"))
)

dcf_sea_rule |>
  filter(Sea_reg != stecf_rule) |>
  arrange(Sea_reg, desc(landed_tonnes)) |>
  select(sub_reg_clean, Sea_reg, landed_tonnes) |>
  gt() |>
  fmt_number(landed_tonnes, decimals = 0) |>
  cols_label(
    sub_reg_clean = "Sub-region",
    Sea_reg = "Assigned to",
    landed_tonnes = "Tonnes, 2002 to 2007"
  )
```

The second check shows what was left out because the country is not in the STECF 2025 panel.

```
dcf_landings_raw |>
  filter(year %in% dcf_years, variable == "totwghtlandg", value > 0) |>
  summarise(landed_tonnes = sum(value) / 1000, .by = c(year, country_code)) |>
  mutate(share_pct = 100 * landed_tonnes / sum(landed_tonnes), .by = year) |>
  filter(!country_code %in% stecf_countries) |>
```

Sub-region	Assigned to	Tonnes, 2002 to 2007
27.3	Baltic Sea	1,258,597
37.4.2	Black Sea	3
37.1	Mediterranean Sea	913,353
37.2.1	Mediterranean Sea	615,496
37.3.1	Mediterranean Sea	407,960
37.2.2	Mediterranean Sea	321,706
37.1.3	Mediterranean Sea	264,575
37.1.2	Mediterranean Sea	132,985
37	Mediterranean Sea	17,655
37.3.2	Mediterranean Sea	5,984
37.2	Mediterranean Sea	1,956
37.3	Mediterranean Sea	9
27.7	NEAO	2,090,009
27.4	NEAO	1,281,273
27.9	NEAO	923,975
27.6	NEAO	558,684
27.2	NEAO	254,953
27.12	NEAO	131,216
27	NEAO	86,000
27.5	NEAO	52,998
27.14	NEAO	42,267
27.10	NEAO	16,614
27.1	NEAO	12,828
27.3.b,c	NEAO	8,421

Year	Country	Tonnes dropped	Share of the year (%)
2002	GBR	654,573	15.6
2003	GBR	598,187	13.5
2004	GBR	626,919	13.1
2005	GBR	688,359	14.4
2006	GBR	600,695	13.3
2007	GBR	600,093	14.6

```
  arrange(country_code, year) |>
  gt() |>
  fmt_number(landed_tonnes, decimals = 0) |>
  fmt_number(share_pct, decimals = 1) |>
  cols_label(
    country_code = "Country", year = "Year",
    landed_tonnes = "Tonnes dropped", share_pct = "Share of the year (%)"
  )
```

The third check shows how much of the catch finds a fleet with a positive operating cash flow, and therefore carries observed rent, and how much goes to the Box 2 gap filling. It also lists the species codes with no typology in the classification workbook.

```
landings_dcf |>
  left_join(fleet_net_dcf, by = join_by(year, fs_name)) |>
  summarise(
    landed_tonnes = sum(landed_kg) / 1000,
    with_ocf_pct = 100 * sum(landed_kg[!is.na(total_net)]) / sum(landed_kg),
    positive_ocf_pct = 100 * sum(landed_kg[total_net > 0], na.rm = TRUE) / sum(landed_kg),
    .by = year
  ) |>
  arrange(year) |>
  gt() |>
  fmt_number(landed_tonnes, decimals = 0) |>
  fmt_number(c(with_ocf_pct, positive_ocf_pct), decimals = 1) |>
  cols_label(
    year = "Year",
    landed_tonnes = "Tonnes",
    with_ocf_pct = "Fleet has OCF (%)",
    positive_ocf_pct = "Fleet OCF positive (%)"
  )
```

Year	Tonnes	Fleet has OCF (%)	Fleet OCF positive (%)
2002	3,528,999	99.6	94.4
2003	3,834,162	84.6	80.9
2004	4,170,734	91.8	84.2
2005	4,082,140	91.8	88.0
2006	3,929,544	91.2	89.9
2007	3,519,848	89.8	88.2

Species code	Tonnes, 2002 to 2007
PIA	147,545
OTHER	12,600
MSX	10,674
UNKNOWN	7,118
ASN	5,655
BBB	1,303
GGG	1,204
ALH	884
WHX	692
PNI	406

```
landings_dcf |>
  filter(is.na(fish_type)) |>
  summarise(landed_tonnes = sum(landed_kg) / 1000, .by = species_code) |>
  slice_max(landed_tonnes, n = 10) |>
  gt() |>
  fmt_number(landed_tonnes, decimals = 0) |>
  cols_label(species_code = "Species code", landed_tonnes = "Tonnes, 2002 to 2007")
```

Overlap with STECF in 2008 and 2009

Both calls report 2008 and 2009, which is the only place the two sources can be compared directly. The same preparation is run on the 2011 call for those two years and set against STECF 2025.

```
overlap_years <- 2008:2009

dcf_landings_raw |>
  clean_dcf_landings(overlap_years) |>
  summarise(dcf_tonnes = sum(landed_kg) / 1000, .by = c(year, Sea_reg)) |>
  left_join(
    landings |>
      filter(year %in% overlap_years) |>
      summarise(stecf_tonnes = sum(landed_kg) / 1000, .by = c(year, Sea_reg)),
    by = join_by(year, Sea_reg)
  ) |>
  mutate(ratio = dcf_tonnes / stecf_tonnes) |>
  arrange(year, Sea_reg) |>
  gt() |>
  fmt_number(c(dcf_tonnes, stecf_tonnes), decimals = 0) |>
  fmt_number(ratio, decimals = 2) |>
  cols_label(
    year = "Year", Sea_reg = "Sea region",
    dcf_tonnes = "2011 call (t)", stecf_tonnes = "STECF 2025 (t)",
    ratio = "Ratio"
  )
```

Year	Sea region	2011 call (t)	STECF 2025 (t)	Ratio
2008	Baltic Sea	628,465	679,049	0.93
2008	Black Sea	6,739	8,083	0.83
2008	Mediterranean Sea	442,902	325,358	1.36
2008	NEAO	1,979,735	2,281,345	0.87
2008	ROW	264,817	859,794	0.31
2009	Baltic Sea	681,875	708,011	0.96
2009	Black Sea	6,548	7,657	0.86
2009	Mediterranean Sea	332,036	341,315	0.97
2009	NEAO	2,076,471	2,266,685	0.92
2009	ROW	304,939	885,665	0.34
2009	unknown	450	NA	NA

```
stecf_items_not_in_ocf <- stecf_economic |>
  filter(
    year %in% overlap_years,
    variable_code %in% c("totdepcost", "totunpaidlab")
  ) |>
  summarise(items_not_in_ocf = sum(value, na.rm = TRUE), .by = year)

dcf_fleet_net(dcf_economic_raw, overlap_years) |>
  summarise(
    ocf = sum(total_net),
    ocf_positive = sum(total_net[total_net > 0]),
    .by = year
  ) |>
  left_join(
    fleet_net |>
      summarise(
        stecf_net = sum(total_net),
        stecf_net_positive = sum(total_net[total_net > 0]),
        .by = year
      ),
    by = "year"
  ) |>
  left_join(stecf_items_not_in_ocf, by = "year") |>
  mutate(
    stecf_net_ocf_basis = stecf_net + items_not_in_ocf,
    across(-year, \(x) x / 1e6)
  ) |>
  select(year, ocf, stecf_net_ocf_basis, stecf_net, ocf_positive, stecf_net_positive) |>
  gt() |>
  fmt_number(-year, decimals = 0) |>
  cols_label(
    year = "Year",
    ocf = "2011 call OCF",
    stecf_net_ocf_basis = "STECF net before capital and unpaid labour",
    stecf_net = "STECF net (Eq. 3)",
    ocf_positive = "OCF, positive fleets",
    stecf_net_positive = "STECF net, positive fleets"
  ) |>
  tab_header(title = "Net economic result, million nominal euro")
```

For landings, the 2011 call is 4 to 13 percent below STECF 2025 in the Baltic and NEAO, and about 70 percent below in ROW, where the distant-water fleets were poorly covered at the time. The Mediterranean is higher in 2008 only because Greece reported to the 2011 call and is missing from STECF 2025 for that

Net economic result, million nominal euro

Year	2011 call OCF	STECF net before capital and unpaid labour	STECF net (Eq. 3)	OCF, positive fleets	STECF net, positive fleets
2008	1,556	1,238	290	1,689	535
2009	1,781	1,289	267	1,889	529

year. The 2002 to 2007 tonnage is therefore a lower bound, mostly in ROW, and part of the step between 2007 and 2008 is the change of source.

For the economic result the break is larger. In 2008 the fleets with a positive operating cash flow add up to about 1.7 billion euro, against about 0.5 billion euro for fleets with a positive Eq. 3 net result. Part of the gap is the two cost items OCF leaves out; part is the coarser fleet definition, which nets loss-making gears against profitable ones inside a country and length class. Resource rent for 2002 to 2007 is therefore an upper bound relative to the 2008 to 2023 series, and the two periods should not be compared as one continuous series without that caveat.

The largest code without a typology is `PIA`, about 150,000 tonnes over the six years. Those landings appear in the tables under "Not identified", as unmatched codes do in the STECF years.

Joining the two sources

The 2002 to 2007 landings and fleet results are appended to the STECF ones in the next chunk, before the allocation. Every grouping key and every gap-filling join contains `year`, so the two sources never mix: a 2005 record can only receive a 2005 fleet result or a 2005 average price.

Resource rent allocation

Matching landings to fleet economics

Landings and fleet economics meet on the fleet segment name and the year. The join is many-to-one because a fleet has one net result but many landing records. The STECF and 2011 DCF inputs are appended here, so everything below runs once over 2002 to 2023. Records that find no positive net result are the gap set of Box 2; they keep their quantities and are valued later from average prices.

```
landings_matched <- bind_rows(landings, landings_dcf) |>
  left_join(
    bind_rows(fleet_net_positive, fleet_net_dcf_positive),
    by = join_by(fs_name, year),
    relationship = "many-to-one"
  )

landings_valued_input <- landings_matched |>
  filter(!is.na(total_net))

landings_without_rent <- landings_matched |>
  filter(is.na(total_net))
```

Distributing the net result

A fleet earns one net result but fishes several sub-regions and several species, and STECF cannot attribute cost or revenue any more finely than the fleet. The assessment therefore distributes the net result in proportion to the quantities landed, in two nested steps: first across the sub-regions the fleet worked (Eq. 4 and Eq. 5), then across the species it landed in each of those sub-regions (Eq. 6 and Eq. 7).

The original script carried these weights as percentages and divided by one hundred again at each use. We keep them as shares, which is arithmetically identical and removes four opportunities for a misplaced factor of one hundred.

```
landings_valued <- landings_valued_input |>
  mutate(
    fleet_landed_kg = sum(landed_kg, na.rm = TRUE),
    .by = c(year, fs_name)
  ) |>
  mutate(
    fleet_area_landed_kg = sum(landed_kg, na.rm = TRUE),
    .by = c(year, fs_name, sub_reg_clean)
  ) |>
  mutate(
    # Eq. 4: share of the fleet's catch taken in this sub-region
    share_fleet_area = fleet_area_landed_kg / fleet_landed_kg,
    # Eq. 5: net result attributable to the fleet in this sub-region
    net_fleet_area = total_net * share_fleet_area,
    # Eq. 6: share of that sub-regional catch made up by this species
    share_species_area = landed_kg / fleet_area_landed_kg,
    # Eq. 7: net result attributable to the species in this sub-region
    net_species = net_fleet_area * share_species_area
  )
```

Observed prices

The accounts report one figure per species, landing country and sub-region, so the fleet dimension is aggregated away here. Dividing the summed net result by the summed quantity gives the implied resource rent per kilogram, which is the price used to value the gap records in the next section.

```
accounting_keys <- c(
  "year", "country_code", "sub_reg_clean", "Sea_reg",
  "species_code", "species_name", "species_group", "fish_type"
)

rent_prices <- landings_valued |>
  summarise(
    sum_landed = sum(landed_kg, na.rm = TRUE),
    sum_revenue = sum(net_species, na.rm = TRUE),
    .by = all_of(accounting_keys)
  ) |>
  mutate(
    price = if_else(sum_landed > 0, sum_revenue / sum_landed, NA_real_),
```

```
    price_basis = "observed rent"
  )
```

> **Two deliberate differences from the original script**
>
> The original produced this table with `mutate()` followed by `distinct()`, which left one row per underlying fleet record rather than one row per accounting key; because the raw `sub_reg` was among the columns kept, sub-regions that had been collapsed by the recode table survived as separate rows carrying the same group total, and the final aggregation added them together. Using `summarise()` over an explicit key list produces exactly one row per group and removes that double counting.
> The original also guarded the division against a zero in the individual record's own net value rather than in the group's total quantity, which set a price to `NA` for records that had a valid group price. The guard here is on `sum_landed`, which is the quantity actually in the denominator.

Gap set

Landings from fleets without a positive net result are aggregated onto the same accounting keys so that the two halves of the dataset share one grain. They arrive with quantities and no revenue.

```
price_gaps <- landings_without_rent |>
  summarise(
    sum_landed = sum(landed_kg, na.rm = TRUE),
    .by = all_of(accounting_keys)
  )
```

Filling the gaps

Box 2 of the methods report fills the gaps in three passes, each one coarser than the last: the average price of the species in the same country, then in the same sub-region, then across Europe. Each pass keeps what it could value and hands the remainder to the next. Anything still unvalued after the third pass has no observed price anywhere in the dataset for that species.

```
# Pass 1: average price of the species in the same landing country
filled_country <- fill_price_gaps(
  price_gaps,
  rent_prices,
  by = c("year", "species_code", "species_name", "country_code"),
  basis = "country average"
)

# Pass 2: average price of the species in the same sub-region
filled_sub_reg <- filled_country |>
  filter(price == 0) |>
  fill_price_gaps(
    rent_prices,
    by = c("year", "species_code", "species_name", "sub_reg_clean"),
    basis = "sub-region average"
  )
```

```
# Pass 3: average price of the species anywhere in the dataset
filled_europe <- filled_sub_reg |>
  filter(price == 0) |>
  fill_price_gaps(
    rent_prices,
    by = c("year", "species_code", "species_name"),
    basis = "European average"
  )
```

The passes overlap by construction, so the first two contribute only the records they actually valued. The third contributes everything that reaches it, including the records it could not value; those carry a quantity and a revenue of zero, which is what keeps the physical account complete even where the economic data does not reach. They are also extracted on their own, because their size is a useful measure of that reach.

```
accounting_records <- bind_rows(
  rent_prices,
  filled_country |> filter(price > 0),
  filled_sub_reg |> filter(price > 0),
  filled_europe |> mutate(price_basis = replace_na(price_basis, "unvalued"))
)

unvalued_records <- filled_europe |>
  filter(price == 0)
```

Final dataset

The four sources are aggregated once more onto the accounting keys, because a species, country and sub-region combination can receive quantities from both a positive-rent fleet and a gap-filled fleet. The final price is the revenue-weighted average that results, and the report's units are tonnes and thousands of euro.

`price_basis` is collapsed into a readable label rather than dropped, so each row of the published table carries the provenance of its own valuation. `data_source` does the same for the data call each year comes from. Note that both unit conversions are applied to the aggregated columns; the original script converted quantities from the aggregate but converted revenue from the pre-aggregation column, so the two published units were computed on different bases.

```
fish_provisioning <- accounting_records |>
  summarise(
    landed_kg = sum(sum_landed, na.rm = TRUE),
    revenue_eur = sum(sum_revenue, na.rm = TRUE),
    price_basis = str_c(sort(unique(price_basis)), collapse = " + "),
    .by = all_of(accounting_keys)
  ) |>
  mutate(
    price_eur_per_kg = if_else(
      landed_kg > 0, revenue_eur / landed_kg, NA_real_
    ),
    landed_tonnes = landed_kg / 1000,
```

```
    revenue_thousand_eur = revenue_eur / 1000,
    data_source = if_else(
      year %in% dcf_years, "DCF 2011 data call", "STECF 2025 data call"
    )
  ) |>
  arrange(year, Sea_reg, country_code, species_code)
```

Deflation is applied here, after the accounts exist, and nowhere else. Eurostat HICP for EU-27, all-items, annual average index, is read from a committed lookup so the site builds without a network call (Eurostat, 2026). The factor is `HICP_t / HICP_base`. Resource rent in constant euro is nominal rent divided by that factor. The base year is 2010.

```
hicp <- read_csv(hicp_file, show_col_types = FALSE)
hicp_base <- hicp$hicp_index[hicp$year == base_year]
if (length(hicp_base) != 1 || is.na(hicp_base) || hicp_base <= 0) {
  stop("HICP lookup has no usable index for base_year = ", base_year)
}

fish_provisioning <- fish_provisioning |>
  left_join(hicp, by = "year") |>
  mutate(
    deflator = hicp_index / hicp_base,
    revenue_eur_constant = revenue_eur / deflator,
    revenue_thousand_eur_constant = revenue_eur_constant / 1000,
    price_eur_per_kg_constant = if_else(
      landed_kg > 0, revenue_eur_constant / landed_kg, NA_real_
    )
  )

hicp |>
  mutate(
    deflator = hicp_index / hicp_base,
    base = if_else(year == base_year, "base", "")
  ) |>
  gt() |>
  cols_label(
    year = "Year",
    hicp_index = "HICP (2015 = 100)",
    deflator = paste0("Deflator (", base_year, " = 1)"),
    base = ""
  ) |>
  fmt_number(hicp_index, decimals = 2) |>
  fmt_number(deflator, decimals = 4) |>
  tab_source_note(
    "Eurostat prc_hicp_aind, EU-27 (from 2020), CP00, annual average index."
  )
```

Overuse is applied here, after allocation and deflation, and is not folded into Eq. 3 to 7. The official SUT records the actual flow. The FSI overlay uses the integer scores in `data/lookup/Overuse_Total.xlsx`, which is the 2018 worked example: scores differ by sea and cover only the assessed species of methods Table

Year	HICP (2015 = 100)	Deflator (2010 = 1)	
2002	77.81	0.8364	
2003	79.59	0.8555	
2004	81.55	0.8766	
2005	83.45	0.8970	
2006	85.38	0.9178	
2007	87.41	0.9396	
2008	90.64	0.9743	
2009	91.35	0.9819	
2010	93.03	1.0000	base
2011	95.69	1.0286	
2012	98.19	1.0555	
2013	99.49	1.0694	
2014	99.89	1.0737	
2015	100.00	1.0749	
2016	100.18	1.0769	
2017	101.74	1.0936	
2018	103.57	1.1133	
2019	105.04	1.1291	
2020	105.76	1.1368	
2021	108.82	1.1697	
2022	118.82	1.2772	
2023	126.38	1.3585	

Eurostat prc_hicp_aind, EU-27 (from 2020), CP00, annual average index.

19. Table 18 converts that integer to an overuse share of actual flow. Unassessed species are left blank, not treated as sustainable. Aquaculture ASI is out of scope.

```
overuse_share_from_score <- function(score) {
  case_when(
    is.na(score) ~ NA_real_,
    score < 0 ~ 1,
    score == 0 ~ NA_real_,
    score == 1 ~ 0.9,
    score == 2 ~ 0.7,
    score == 3 ~ 0.5,
    score == 4 ~ 0.4,
    score == 5 ~ 0.3,
    score == 6 ~ 0.2,
    score == 7 ~ 0.1,
    score >= 8 ~ 0
  )
}

read_overuse_sheet <- function(sheet, sea) {
  read_excel(overuse_lookup_file, sheet = sheet) |>
    rename(species_code = 1) |>
    transmute(
      Sea_reg = sea,
      species_code,
      fsi_common_name = `Common name`,
      fsi_initial_score = `Initial Score`,
      fsi_score = Score,
      fsi_risk_lookup = `Risk level`
    )
}

overuse_scores <- bind_rows(
  read_overuse_sheet("Baltic Sea", "Baltic Sea"),
  read_overuse_sheet("Black Sea", "Black Sea"),
  read_overuse_sheet("Med Sea", "Mediterranean Sea"),
  read_overuse_sheet("NEAO", "NEAO")
```

```
) |>
  mutate(overuse_share = overuse_share_from_score(fsi_score))

fish_provisioning <- fish_provisioning |>
  left_join(overuse_scores, by = c("Sea_reg", "species_code")) |>
  mutate(
    overuse_tonnes = if_else(
      !is.na(overuse_share), landed_tonnes * overuse_share, NA_real_
    ),
    sustainable_tonnes = if_else(
      !is.na(overuse_share), landed_tonnes * (1 - overuse_share), NA_real_
    ),
    overuse_thousand_eur = if_else(
      !is.na(overuse_share), revenue_thousand_eur * overuse_share, NA_real_
    ),
    overuse_thousand_eur_constant = if_else(
      !is.na(overuse_share),
      revenue_thousand_eur_constant * overuse_share,
      NA_real_
    )
  )

overuse_by_sea_year <- fish_provisioning |>
  filter(!is.na(overuse_share)) |>
  summarise(
    assessed_tonnes = sum(landed_tonnes),
    overuse_tonnes = sum(overuse_tonnes),
    assessed_thousand_eur = sum(revenue_thousand_eur),
    overuse_thousand_eur = sum(overuse_thousand_eur),
    assessed_thousand_eur_constant = sum(revenue_thousand_eur_constant),
    overuse_thousand_eur_constant = sum(overuse_thousand_eur_constant),
    .by = c(year, Sea_reg)
  ) |>
  mutate(
    share_tonnes = overuse_tonnes / assessed_tonnes,
    share_eur = overuse_thousand_eur / assessed_thousand_eur
  )

# Does not fit the PDF page, so shown in HTML only
if (!knitr::is_latex_output()) {
  overuse_by_sea_year |>
    filter(year == 2018) |>
    arrange(Sea_reg) |>
    gt() |>
    fmt_number(
      c(
        assessed_tonnes, overuse_tonnes,
        assessed_thousand_eur, overuse_thousand_eur
      ),
      decimals = 2
```

```
    ) |>
    fmt_percent(c(share_tonnes, share_eur), decimals = 0) |>
    tab_header(title = "Overuse overlay, assessed species only, 2018") |>
    tab_source_note(
      "Shares apply only to species with an FSI score in that sea. This is not a column of the off
    )
}
```

Accounting tables

The methods report prints nested supply and use tables, not the compact sea-by-country summaries. Supply rows are FAO/GFCM fishing areas (`sub_reg_clean`), labelled EEZ as in the report; they are the spatial unit of supply, not legal exclusive economic zones. Use rows are the Member State of landing. Columns are `species_group` crossed with `fish_type`, produced with `pivot_wider()` so the table comes out of the pipeline in the same shape as methods Tables 10 to 13. Unmatched species sit in Others / Not identified. Empty fish-type combinations are filled with zero. Supply totals equal use totals at every grain. The 2023 Baltic tables below are the worked example of chapter 4; the Excel workbook repeats them for every sea and year, in tonnes, thousand nominal euro, and thousand 2010 euro.

```
sea_order <- c(
  "Baltic Sea", "Black Sea", "Mediterranean Sea", "NEAO", "ROW", "unknown"
)

sut_value_cols <- c(
  "fish_benthopelagic", "fish_demersal", "fish_pelagic",
  "shellfish_benthic", "shellfish_benthopelagic",
  "shellfish_demersal", "shellfish_pelagic",
  "other_not_identified"
)

sut_type_labels <- c(
  fish_benthopelagic = "benthopelagic",
  fish_demersal = "demersal",
  fish_pelagic = "pelagic",
  shellfish_benthic = "benthic",
  shellfish_benthopelagic = "benthopelagic",
  shellfish_demersal = "demersal",
  shellfish_pelagic = "pelagic",
  other_not_identified = "Not identified",
  Total = "Total"
)

prepare_sut_types <- function(data) {
  data |>
    mutate(
      species_group = case_when(
        species_group == "fish" ~ "fish",
        species_group == "shellfish" ~ "shellfish",
        .default = "other"
```

```
      ),
      fish_type = case_when(
        species_group == "other" ~ "not_identified",
        is.na(fish_type) ~ "not_identified",
        .default = fish_type
      ),
      species_group = factor(species_group, c("fish", "shellfish", "other")),
      fish_type = factor(
        fish_type,
        c("benthic", "benthopelagic", "demersal", "pelagic", "not_identified")
      )
    )
}

nested_sut <- function(data, row_var, value_var) {
  wide <- data |>
    prepare_sut_types() |>
    summarise(
      value = sum(.data[[value_var]], na.rm = TRUE),
      .by = all_of(c(row_var, "species_group", "fish_type"))
    ) |>
    pivot_wider(
      names_from = c(species_group, fish_type),
      values_from = value,
      names_sep = "_",
      values_fill = 0,
      names_expand = TRUE
    )

  missing_cols <- setdiff(sut_value_cols, names(wide))
  for (col in missing_cols) {
    wide[[col]] <- 0
  }

  wide |>
    mutate(Total = rowSums(across(all_of(sut_value_cols)), na.rm = TRUE)) |>
    filter(Total != 0) |>
    select(all_of(row_var), all_of(sut_value_cols), Total) |>
    arrange(across(all_of(row_var)))
}

add_total_row <- function(tbl, row_var) {
  totals <- tbl |>
    summarise(across(where(is.numeric), \(x) sum(x, na.rm = TRUE))) |>
    mutate(!!row_var := "Total", .before = 1)
  bind_rows(tbl, totals)
}

sea_sut <- function(year, sea, side, value_var, data = fish_provisioning) {
  slice <- data |> filter(year == .env$year, Sea_reg == .env$sea)
```

```
  if (side == "supply") {
    nested_sut(slice, "sub_reg_clean", value_var) |>
      add_total_row("sub_reg_clean") |>
      rename(EEZ = sub_reg_clean)
  } else {
    slice |>
      left_join(country_names, by = "country_code") |>
      mutate(country = coalesce(country, country_code)) |>
      nested_sut("country", value_var) |>
      add_total_row("country") |>
      rename(Country = country)
  }
}

europe_supply_sut <- function(year, value_var, data = fish_provisioning) {
  data |>
    filter(year == .env$year) |>
    nested_sut("Sea_reg", value_var) |>
    mutate(Sea_reg = factor(Sea_reg, levels = sea_order)) |>
    arrange(Sea_reg) |>
    mutate(Sea_reg = as.character(Sea_reg)) |>
    add_total_row("Sea_reg") |>
    rename(`Fishing region` = Sea_reg)
}

europe_use_sut <- function(year, value_var, data = fish_provisioning) {
  data |>
    filter(year == .env$year) |>
    left_join(country_names, by = "country_code") |>
    mutate(country = coalesce(country, country_code)) |>
    nested_sut("country", value_var) |>
    add_total_row("country") |>
    rename(Country = country)
}

europe_origin_sut <- function(year, value_var, data = fish_provisioning) {
  data |>
    filter(year == .env$year) |>
    left_join(country_names, by = "country_code") |>
    mutate(country = coalesce(country, country_code)) |>
    summarise(value = sum(.data[[value_var]], na.rm = TRUE), .by = c(country, Sea_reg)) |>
    mutate(Sea_reg = factor(Sea_reg, levels = sea_order)) |>
    pivot_wider(
      names_from = Sea_reg,
      values_from = value,
      values_fill = 0,
      names_expand = TRUE
    ) |>
    mutate(Total = rowSums(across(all_of(sea_order)), na.rm = TRUE)) |>
    arrange(country) |>
```

```
    add_total_row("country") |>
    rename(Country = country)
}

sut_gt <- function(tbl, title, note) {
  row_col <- names(tbl)[1]
  tbl |>
    gt(rowname_col = row_col) |>
    cols_label(.list = as.list(sut_type_labels)) |>
    tab_spanner("Fish", c(fish_benthopelagic, fish_demersal, fish_pelagic)) |>
    tab_spanner(
      "Shellfish",
      c(
        shellfish_benthic, shellfish_benthopelagic,
        shellfish_demersal, shellfish_pelagic
      )
    ) |>
    tab_spanner("Others", other_not_identified) |>
    fmt_number(decimals = 2) |>
    tab_header(title = title) |>
    tab_source_note(note) |>
    tab_stubhead(label = row_col) |>
    # Ten columns overflow a portrait PDF page at the default size
    tab_options(table.font.size = if (knitr::is_latex_output()) px(7) else NULL)
}

sut_identity <- function(supply, use) {
  abs(dplyr::last(supply$Total) - dplyr::last(use$Total)) < 0.05
}

eez_note <- paste(
  "EEZ is the FAO/GFCM fishing area used as the spatial unit of supply,",
  "not a legal exclusive economic zone. Country is the Member State of landing."
)
```

```
baltic_supply_t <- sea_sut(reference_year, "Baltic Sea", "supply", "landed_tonnes")
baltic_use_t <- sea_sut(reference_year, "Baltic Sea", "use", "landed_tonnes")
baltic_supply_nom <- sea_sut(reference_year, "Baltic Sea", "supply", "revenue_thousand_eur")
baltic_use_nom <- sea_sut(reference_year, "Baltic Sea", "use", "revenue_thousand_eur")
baltic_supply_const <- sea_sut(
  reference_year, "Baltic Sea", "supply", "revenue_thousand_eur_constant"
)
baltic_use_const <- sea_sut(
  reference_year, "Baltic Sea", "use", "revenue_thousand_eur_constant"
)

stopifnot(
  sut_identity(baltic_supply_t, baltic_use_t),
  sut_identity(baltic_supply_nom, baltic_use_nom),
  sut_identity(baltic_supply_const, baltic_use_const)
```

Table 10. Baltic Sea supply, physical (t), 2023

	Fish			Shellfish				Others	
EEZ	benthopelagic	demersal	pelagic	benthic	benthopelagic	demersal	pelagic	Not identified	Total
27.3.d	0.00	0.00	0.50	0.00	0.00	0.00	0.00	0.00	0.50
27.3.d.24	1,590.70	2,878.05	2,137.66	10.07	0.00	0.00	0.00	0.02	6,616.50
27.3.d.25	1,909.49	7,169.80	53,013.70	0.00	0.00	0.00	0.00	2.18	62,095.16
27.3.d.26	1,735.30	1,130.42	82,396.76	0.00	0.00	0.00	0.00	0.00	85,262.49
27.3.d.27	26.30	44.53	20,232.49	0.00	0.00	0.00	0.00	0.00	20,303.33
27.3.d.28	551.46	2,186.20	119,645.74	0.05	0.00	0.00	0.00	47.90	122,431.35
27.3.d.29	326.75	771.71	35,596.02	0.00	0.00	0.00	0.00	23.71	36,718.18
27.3.d.30	1,033.42	711.38	69,975.13	0.00	0.00	0.00	0.00	0.00	71,719.93
27.3.d.31	2,206.83	316.61	848.54	0.00	0.00	0.00	0.00	0.00	3,371.97
27.3.d.32	75.03	104.50	27,970.31	0.00	0.00	0.00	0.00	2.69	28,152.53
Total	9,455.28	15,313.19	411,816.84	10.12	0.00	0.00	0.00	76.49	436,671.93

EEZ is the FAO/GFCM fishing area used as the spatial unit of supply, not a legal exclusive economic zone. Country is the Member State of landing.

Table 12. Baltic Sea use, physical (t), 2023

	Fish			Shellfish				Others	
Country	benthopelagic	demersal	pelagic	benthic	benthopelagic	demersal	pelagic	Not identified	Total
Denmark	58.22	691.99	29,412.06	7.89	0.00	0.00	0.00	0.02	30,170.18
Estonia	414.70	1,324.59	55,677.20	0.00	0.00	0.00	0.00	74.29	57,490.78
Finland	2,019.12	1,267.77	86,384.52	0.00	0.00	0.00	0.00	0.00	89,671.41
Germany	690.89	850.45	13,196.09	2.18	0.00	0.00	0.00	0.00	14,739.62
Latvia	304.82	1,456.50	59,684.88	0.00	0.00	0.00	0.00	0.00	61,446.19
Lithuania	32.23	105.98	12,877.56	0.00	0.00	0.00	0.00	0.00	13,015.78
Poland	4,496.15	9,406.15	81,481.28	0.00	0.00	0.00	0.00	2.18	95,385.76
Sweden	1,439.16	209.76	73,103.25	0.05	0.00	0.00	0.00	0.00	74,752.22
Total	9,455.28	15,313.19	411,816.84	10.12	0.00	0.00	0.00	76.49	436,671.93

EEZ is the FAO/GFCM fishing area used as the spatial unit of supply, not a legal exclusive economic zone. Country is the Member State of landing.

```
)

sut_gt(
  baltic_supply_t,
  paste0("Table 10. Baltic Sea supply, physical (t), ", reference_year),
  eez_note
)
```

```
sut_gt(
  baltic_use_t,
  paste0("Table 12. Baltic Sea use, physical (t), ", reference_year),
  eez_note
)
```

```
sut_gt(
  baltic_supply_nom,
  paste0("Table 11. Baltic Sea supply, thousand nominal euro, ", reference_year),
  eez_note
)
```

```
sut_gt(
  baltic_use_nom,
  paste0("Table 13. Baltic Sea use, thousand nominal euro, ", reference_year),
  eez_note
)
```

Table 11. Baltic Sea supply, thousand nominal euro, 2023

	Fish			Shellfish				Others	
EEZ	benthopelagic	demersal	pelagic	benthic	benthopelagic	demersal	pelagic	Not identified	Total
27.3.d	0.00	0.00	0.11	0.00	0.00	0.00	0.00	0.00	0.11
27.3.d.24	657.40	652.57	494.64	4.84	0.00	0.00	0.00	0.00	1,809.44
27.3.d.25	285.16	1,115.43	7,579.42	0.00	0.00	0.00	0.00	0.28	8,980.29
27.3.d.26	809.02	395.02	11,129.67	0.00	0.00	0.00	0.00	0.00	12,333.71
27.3.d.27	5.47	64.18	3,153.80	0.00	0.00	0.00	0.00	0.00	3,223.45
27.3.d.28	114.95	539.66	21,335.87	0.01	0.00	0.00	0.00	6.37	21,996.85
27.3.d.29	35.15	75.35	2,691.16	0.00	0.00	0.00	0.00	3.03	2,804.69
27.3.d.30	146.57	22.48	2,519.06	0.00	0.00	0.00	0.00	0.00	2,688.11
27.3.d.31	853.69	16.10	137.58	0.00	0.00	0.00	0.00	0.00	1,007.36
27.3.d.32	7.21	7.82	3,096.57	0.00	0.00	0.00	0.00	0.31	3,111.91
Total	2,914.61	2,888.60	52,137.88	4.85	0.00	0.00	0.00	9.99	57,955.94

EEZ is the FAO/GFCM fishing area used as the spatial unit of supply, not a legal exclusive economic zone. Country is the Member State of landing.

Table 13. Baltic Sea use, thousand nominal euro, 2023

	Fish			Shellfish				Others	
Country	benthopelagic	demersal	pelagic	benthic	benthopelagic	demersal	pelagic	Not identified	Total
Denmark	4.40	84.43	2,209.90	4.22	0.00	0.00	0.00	0.00	2,302.94
Estonia	58.60	182.05	8,615.49	0.00	0.00	0.00	0.00	9.71	8,865.84
Finland	245.82	22.94	1,507.25	0.00	0.00	0.00	0.00	0.00	1,776.01
Germany	255.98	322.66	736.58	0.62	0.00	0.00	0.00	0.00	1,315.84
Latvia	79.51	454.27	13,761.98	0.00	0.00	0.00	0.00	0.00	14,295.76
Lithuania	9.33	44.89	1,341.92	0.00	0.00	0.00	0.00	0.00	1,396.14
Poland	1,484.06	1,653.65	12,716.35	0.00	0.00	0.00	0.00	0.28	15,854.34
Sweden	776.92	123.72	11,248.41	0.01	0.00	0.00	0.00	0.00	12,149.06
Total	2,914.61	2,888.60	52,137.88	4.85	0.00	0.00	0.00	9.99	57,955.94

EEZ is the FAO/GFCM fishing area used as the spatial unit of supply, not a legal exclusive economic zone. Country is the Member State of landing.

```
sut_gt(
  baltic_supply_const,
  paste0(
    "Baltic Sea supply, thousand ", base_year, " euro, ", reference_year
  ),
  eez_note
)
```

```
sut_gt(
  baltic_use_const,
  paste0("Baltic Sea use, thousand ", base_year, " euro, ", reference_year),
  eez_note
)
```

Baltic Sea supply, thousand 2010 euro, 2023

	Fish			Shellfish				Others	
EEZ	benthopelagic	demersal	pelagic	benthic	benthopelagic	demersal	pelagic	Not identified	Total
27.3.d	0.00	0.00	0.08	0.00	0.00	0.00	0.00	0.00	0.08
27.3.d.24	483.92	480.36	364.11	3.56	0.00	0.00	0.00	0.00	1,331.95
27.3.d.25	209.91	821.08	5,579.31	0.00	0.00	0.00	0.00	0.21	6,610.51
27.3.d.26	595.53	290.78	8,192.70	0.00	0.00	0.00	0.00	0.00	9,079.01
27.3.d.27	4.02	47.25	2,321.56	0.00	0.00	0.00	0.00	0.00	2,372.83
27.3.d.28	84.61	397.25	15,705.62	0.01	0.00	0.00	0.00	4.69	16,192.18
27.3.d.29	25.87	55.47	1,981.00	0.00	0.00	0.00	0.00	2.23	2,064.57
27.3.d.30	107.89	16.55	1,854.31	0.00	0.00	0.00	0.00	0.00	1,978.75
27.3.d.31	628.41	11.85	101.27	0.00	0.00	0.00	0.00	0.00	741.53
27.3.d.32	5.31	5.75	2,279.43	0.00	0.00	0.00	0.00	0.23	2,290.72
Total	2,145.49	2,126.34	38,379.39	3.57	0.00	0.00	0.00	7.35	42,662.14

EEZ is the FAO/GFCM fishing area used as the spatial unit of supply, not a legal exclusive economic zone. Country is the Member State of landing.

Baltic Sea use, thousand 2010 euro, 2023

	Fish			Shellfish				Others	
Country	benthopelagic	demersal	pelagic	benthic	benthopelagic	demersal	pelagic	Not identified	Total
Denmark	3.24	62.15	1,626.74	3.11	0.00	0.00	0.00	0.00	1,695.23
Estonia	43.14	134.01	6,341.97	0.00	0.00	0.00	0.00	7.15	6,526.27
Finland	180.95	16.89	1,109.51	0.00	0.00	0.00	0.00	0.00	1,307.35
Germany	188.43	237.51	542.21	0.46	0.00	0.00	0.00	0.00	968.61
Latvia	58.53	334.40	10,130.38	0.00	0.00	0.00	0.00	0.00	10,523.30
Lithuania	6.86	33.04	987.81	0.00	0.00	0.00	0.00	0.00	1,027.72
Poland	1,092.44	1,217.27	9,360.67	0.00	0.00	0.00	0.00	0.21	11,670.59
Sweden	571.90	91.07	8,280.11	0.01	0.00	0.00	0.00	0.00	8,943.09
Total	2,145.49	2,126.34	38,379.39	3.57	0.00	0.00	0.00	7.35	42,662.14

EEZ is the FAO/GFCM fishing area used as the spatial unit of supply, not a legal exclusive economic zone. Country is the Member State of landing.

Outputs

The original script wrote twelve intermediate workbooks to disk. We write the panel twice as a flat deliverable, once as RDS and once as Excel, so the client can open the same table in either tool. Observed prices and unvalued catch stay as RDS. The third workbook is the report-style SUT file: one sheet per year, tables stacked with a blank row between them. Each nested table has the column layout of methods Tables 10 to 13. Europe-wide Tables 2 to 9 sit at the top of the sheet; the per-sea nested SUTs follow. A Checks sheet records the supply-use identity at every grain.

```
saveRDS(fish_provisioning, file.path(results_dir, "fish_provisioning.RDS"))
saveRDS(rent_prices, file.path(results_dir, "rent_prices.RDS"))
saveRDS(unvalued_records, file.path(results_dir, "unvalued_records.RDS"))

writexl::write_xlsx(
  list(fish_provisioning = fish_provisioning),
  path = file.path(results_dir, "fish_provisioning.xlsx")
)

result_years <- sort(unique(fish_provisioning$year), decreasing = TRUE)
nested_seas <- sea_order
col_width <- 10
number_format <- "#,##0.00"

metric_specs <- tribble(
  ~key, ~value_var, ~unit_label,
  "physical", "landed_tonnes", "physical terms (t)",
  "nominal", "revenue_thousand_eur", "monetary terms (1,000 EUR, current prices)",
  "constant", "revenue_thousand_eur_constant",
  paste0("monetary terms (1,000 EUR, constant ", base_year, " prices)")
)

europe_region_table <- function(year, value_var) {
  by_sea <- fish_provisioning |>
    filter(year == .env$year) |>
    summarise(value = sum(.data[[value_var]], na.rm = TRUE), .by = Sea_reg) |>
    mutate(Sea_reg = factor(Sea_reg, levels = sea_order)) |>
    complete(Sea_reg, fill = list(value = 0)) |>
    arrange(Sea_reg) |>
    mutate(Sea_reg = as.character(Sea_reg))
  supply <- by_sea |>
```

```
    pivot_wider(names_from = Sea_reg, values_from = value) |>
    mutate(Total = rowSums(across(everything()), na.rm = TRUE)) |>
    mutate(SUT = "Supply", .before = 1)
  use <- supply |>
    mutate(SUT = "Use") |>
    mutate(across(-c(SUT, Total), \(v) NA_real_))
  bind_rows(supply, use)
}

write_banner_block <- function(wb, sheet, start_row, n_cols, title, year, grain) {
  lines <- c(
    "European Commission, Joint Research Centre (JRC)",
    title,
    paste0(
      "Year ", year, ". ", grain,
      " FAO/GFCM fishing areas are labelled EEZ as the spatial unit of supply; ",
      "they are not legal exclusive economic zones. Constant prices use ",
      base_year, " as the base year (Eurostat HICP, EU-27)."
    )
  )
  for (i in 1:3) {
    row <- start_row + i - 1L
    wb <- wb_add_data(
      wb, sheet, x = lines[[i]],
      start_row = row, start_col = 1, col_names = FALSE
    )
    wb <- wb_merge_cells(wb, sheet, dims = wb_dims(rows = row, cols = 1:n_cols))
    wb <- wb_add_cell_style(
      wb, sheet,
      dims = wb_dims(rows = row, cols = 1),
      wrap_text = TRUE, vertical = "center"
    )
  }
  wb_set_row_heights(wb, sheet, rows = start_row:(start_row + 2L), heights = 18)
}

write_nested_headers <- function(wb, sheet, header_row, row_label) {
  group_row <- header_row
  type_row <- header_row + 1L
  group_vals <- c(
    row_label, "Fish", "Fish", "Fish",
    "Shellfish", "Shellfish", "Shellfish", "Shellfish",
    "Others", "Total"
  )
  type_vals <- c(
    row_label, "benthopelagic", "demersal", "pelagic",
    "benthic", "benthopelagic", "demersal", "pelagic",
    "Not identified", "Total"
  )
  wb <- wb_add_data(
```

```
    wb, sheet, x = matrix(group_vals, nrow = 1),
    start_row = group_row, start_col = 1, col_names = FALSE
  )
  wb <- wb_add_data(
    wb, sheet, x = matrix(type_vals, nrow = 1),
    start_row = type_row, start_col = 1, col_names = FALSE
  )
  wb <- wb_merge_cells(wb, sheet, dims = wb_dims(rows = group_row:type_row, cols = 1))
  wb <- wb_merge_cells(wb, sheet, dims = wb_dims(rows = group_row, cols = 2:4))
  wb <- wb_merge_cells(wb, sheet, dims = wb_dims(rows = group_row, cols = 5:8))
  header_dims <- wb_dims(rows = group_row:type_row, cols = 1:10)
  wb <- wb_add_font(wb, sheet, dims = header_dims, bold = TRUE)
  wb <- wb_add_cell_style(
    wb, sheet, dims = header_dims,
    wrap_text = TRUE, horizontal = "center", vertical = "center"
  )
  wb <- wb_add_border(
    wb, sheet,
    dims = wb_dims(rows = type_row, cols = 1:10),
    bottom_border = "thin",
    top_border = "none",
    left_border = "none",
    right_border = "none"
  )
  wb <- wb_set_row_heights(wb, sheet, rows = group_row:type_row, heights = 22)
  wb
}

write_data_block <- function(wb, sheet, data, data_start) {
  n_cols <- ncol(data)
  data_end <- data_start + nrow(data) - 1L
  wb <- wb_add_data(
    wb, sheet, x = data,
    start_row = data_start, start_col = 1,
    col_names = FALSE, na = "", apply_cell_style = FALSE
  )
  wb <- wb_add_numfmt(
    wb, sheet,
    dims = wb_dims(rows = data_start:data_end, cols = 2:n_cols),
    numfmt = number_format
  )
  total_row <- data_end
  total_dims <- wb_dims(rows = total_row, cols = 1:n_cols)
  wb <- wb_add_font(wb, sheet, dims = total_dims, bold = TRUE)
  wb <- wb_add_border(
    wb, sheet, dims = total_dims,
    top_border = "thin",
    bottom_border = "thin",
    left_border = "none",
    right_border = "none"
```

```
  )
  wb <- wb_set_row_heights(wb, sheet, rows = total_row, heights = 18)
  wb <- wb_set_col_widths(wb, sheet, cols = 1:n_cols, widths = col_width)
  wb
}

write_nested_table <- function(wb, sheet, start_row, title, year, grain, data) {
  n_cols <- ncol(data)
  wb <- write_banner_block(wb, sheet, start_row, n_cols, title, year, grain)
  header_row <- start_row + 4L
  wb <- write_nested_headers(wb, sheet, header_row, names(data)[1])
  data_start <- header_row + 2L
  wb <- write_data_block(wb, sheet, data, data_start)
  list(wb = wb, next_row = data_start + nrow(data) + 1L)
}

write_simple_table <- function(wb, sheet, start_row, title, year, grain, data) {
  n_cols <- ncol(data)
  wb <- write_banner_block(wb, sheet, start_row, n_cols, title, year, grain)
  header_row <- start_row + 4L
  headers <- names(data)
  wb <- wb_add_data(
    wb, sheet, x = matrix(headers, nrow = 1),
    start_row = header_row, start_col = 1, col_names = FALSE
  )
  header_dims <- wb_dims(rows = header_row, cols = 1:n_cols)
  wb <- wb_add_font(wb, sheet, dims = header_dims, bold = TRUE)
  wb <- wb_add_cell_style(
    wb, sheet, dims = header_dims,
    wrap_text = TRUE, horizontal = "center", vertical = "center"
  )
  wb <- wb_add_border(
    wb, sheet, dims = header_dims,
    bottom_border = "thin",
    top_border = "none",
    left_border = "none",
    right_border = "none"
  )
  wb <- wb_set_row_heights(wb, sheet, rows = header_row, heights = 36)
  wb <- write_data_block(wb, sheet, data, header_row + 1L)
  list(wb = wb, next_row = header_row + 1L + nrow(data) + 1L)
}

identity_rows <- list()
append_identity <- function(year, grain, metric, supply, use) {
  identity_rows[[length(identity_rows) + 1L]] <<- tibble(
    year = year,
    grain = grain,
    metric = metric,
    supply_total = dplyr::last(supply$Total),
```

```
    use_total = dplyr::last(use$Total),
    difference = dplyr::last(supply$Total) - dplyr::last(use$Total),
    match = abs(dplyr::last(supply$Total) - dplyr::last(use$Total)) < 0.05
  )
}

wb <- wb_workbook()
wb <- wb_add_worksheet(wb, "Readme")
readme_notes <- c(
  "European Commission, Joint Research Centre (JRC)",
  "Fish provisioning ecosystem service accounts, supply and use tables",
  paste0(
    "Base year ", base_year,
    ". Resource rent in thousand euro. Constant prices use Eurostat HICP for EU-27."
  ),
  "One sheet per year. Tables are stacked, with at least one blank row between them.",
  "Nested columns follow methods Tables 4 to 7 and 10 to 13: Fish (benthopelagic, demersal, pelagi
  "Supply rows labelled EEZ are FAO/GFCM fishing areas (sub_reg_clean), the spatial unit of supply
  "Use rows are the STECF Member State of landing, shown as the ISO name.",
  "Europe tables at the top of each year sheet correspond to methods Tables 2 to 9. Per-sea nested
  "Supply Total equals use Total at every grain; see the Checks sheet.",
  "Overuse is a sustainability overlay on assessed species only. It is not a column of the officia
  "The machine-readable panel is fish_provisioning.xlsx / fish_provisioning.RDS.",
  "Source: STECF 2025 Fleet Economic data call for 2008 to 2023.",
  paste(
    "Source for 2002 to 2007: 2011 DCF data call. Fleets are country plus vessel length,",
    "the net result is operating cash flow, the United Kingdom is excluded, and fishing areas",
    "are coarser (Mediterranean in FAO divisions). Compare across 2007 and 2008 with care."
  )
)
wb <- wb_add_data(wb, "Readme", x = readme_notes, col_names = FALSE)
wb <- wb_add_cell_style(
  wb, "Readme",
  dims = wb_dims(rows = seq_along(readme_notes), cols = 1),
  wrap_text = TRUE
)
wb <- wb_set_col_widths(wb, "Readme", cols = 1, widths = 80)

for (y in result_years) {
  year_name <- as.character(y)
  wb <- wb_add_worksheet(wb, year_name)
  current_row <- 1L

  for (i in seq_len(nrow(metric_specs))) {
    spec <- metric_specs[i, ]
    region_tbl <- europe_region_table(y, spec$value_var)
    written <- write_simple_table(
      wb, year_name, current_row,
      paste0("SUT of fish provisioning in ", spec$unit_label),
      y,
```

```
    "Europe. Fishing region as ecosystem type; use is the fisheries / primary-sector total.",
    region_tbl
  )
  wb <- written$wb
  current_row <- written$next_row

  europe_supply <- europe_supply_sut(y, spec$value_var)
  europe_use <- europe_use_sut(y, spec$value_var)
  append_identity(y, "Europe nested", spec$key, europe_supply, europe_use)

  written <- write_nested_table(
    wb, year_name, current_row,
    paste0("Supply table of fish provisioning in ", spec$unit_label, ", per fish type"),
    y,
    "Europe. Rows are fishing regions.",
    europe_supply
  )
  wb <- written$wb
  current_row <- written$next_row

  written <- write_nested_table(
    wb, year_name, current_row,
    paste0("Use table of fish provisioning in ", spec$unit_label, ", per fish type"),
    y,
    "Europe. Rows are landing countries.",
    europe_use
  )
  wb <- written$wb
  current_row <- written$next_row

  origin_tbl <- europe_origin_sut(y, spec$value_var)
  written <- write_simple_table(
    wb, year_name, current_row,
    paste0("Use table of fish provisioning in ", spec$unit_label, ", per fishing region"),
    y,
    "Europe. Landing country by fishing region of origin.",
    origin_tbl
  )
  wb <- written$wb
  current_row <- written$next_row
}

for (sea in nested_seas) {
  sea_rows <- fish_provisioning$Sea_reg == sea & fish_provisioning$year == y
  if (!any(sea_rows)) {
    next
  }
  for (i in seq_len(nrow(metric_specs))) {
    spec <- metric_specs[i, ]
    supply_tbl <- sea_sut(y, sea, "supply", spec$value_var)
```

```
      use_tbl <- sea_sut(y, sea, "use", spec$value_var)
      append_identity(y, sea, spec$key, supply_tbl, use_tbl)

      written <- write_nested_table(
        wb, year_name, current_row,
        paste0(
          "Supply table of fish provisioning in ", sea, " in ",
          spec$unit_label, " per fish type"
        ),
        y,
        paste0(sea, ". Supply rows are FAO/GFCM fishing areas, labelled EEZ."),
        supply_tbl
      )
      wb <- written$wb
      current_row <- written$next_row

      written <- write_nested_table(
        wb, year_name, current_row,
        paste0(
          "Use table of fish provisioning in ", sea, " in ",
          spec$unit_label, " per fish type"
        ),
        y,
        paste0(sea, ". Use rows are landing countries."),
        use_tbl
      )
      wb <- written$wb
      current_row <- written$next_row
    }
  }

  wb <- wb_freeze_pane(
    wb, year_name,
    first_active_row = 6L, first_active_col = 2L
  )
}

identity_tbl <- bind_rows(identity_rows)
stopifnot(
  "Supply Total must equal use Total at every SUT grain" = all(identity_tbl$match)
)

wb <- wb_add_worksheet(wb, "Checks")
wb <- wb_add_data(wb, "Checks", x = identity_tbl)
wb <- wb_add_numfmt(
  wb, "Checks",
  dims = wb_dims(rows = 2:(nrow(identity_tbl) + 1L), cols = 4:6),
  numfmt = number_format
)
wb <- wb_set_col_widths(wb, "Checks", cols = 1:7, widths = 18)
```

```r
overuse_species <- fish_provisioning |>
  filter(!is.na(overuse_share)) |>
  summarise(
    landed_tonnes = sum(landed_tonnes),
    overuse_tonnes = sum(overuse_tonnes),
    sustainable_tonnes = sum(sustainable_tonnes),
    revenue_thousand_eur = sum(revenue_thousand_eur),
    overuse_thousand_eur = sum(overuse_thousand_eur),
    revenue_thousand_eur_constant = sum(revenue_thousand_eur_constant),
    overuse_thousand_eur_constant = sum(overuse_thousand_eur_constant),
    fsi_score = dplyr::first(fsi_score),
    overuse_share = dplyr::first(overuse_share),
    fsi_common_name = dplyr::first(fsi_common_name),
    .by = c(year, Sea_reg, species_code)
  ) |>
  arrange(year, Sea_reg, species_code)

wb <- wb_add_worksheet(wb, "Overuse")
wb <- wb_add_data(
  wb, "Overuse",
  x = c(
    "Overuse overlay on assessed species only. Not part of the official SUT.",
    "FSI scores are the 2018 worked example in data/lookup/Overuse_Total.xlsx and are held constan
  ),
  col_names = FALSE
)
wb <- wb_add_data(wb, "Overuse", x = overuse_species, start_row = 4)
wb <- wb_set_col_widths(wb, "Overuse", cols = 1:12, widths = 16)

wb <- wb_add_worksheet(wb, "Comparison")
cmp_physical <- europe_supply_sut(2018, "landed_tonnes") |>
  select(`Fishing region`, Total_2018 = Total) |>
  left_join(
    europe_supply_sut(2023, "landed_tonnes") |>
      select(`Fishing region`, Total_2023 = Total),
    by = "Fishing region"
  )
cmp_constant <- europe_supply_sut(2018, "revenue_thousand_eur_constant") |>
  select(`Fishing region`, Total_2018 = Total) |>
  left_join(
    europe_supply_sut(2023, "revenue_thousand_eur_constant") |>
      select(`Fishing region`, Total_2023 = Total),
    by = "Fishing region"
  )
wb <- wb_add_data(
  wb, "Comparison",
  x = paste0(
    "Tables 14 to 15 analogue. Physical tonnes and thousand ",
    base_year, " euro. Supply totals; use equals supply at this grain."
  ),
```

```
  col_names = FALSE
)
wb <- wb_add_data(wb, "Comparison", x = "Physical (t)", start_row = 3, col_names = FALSE)
wb <- wb_add_data(wb, "Comparison", x = cmp_physical, start_row = 4)
wb <- wb_add_data(
  wb, "Comparison",
  x = paste0("Thousand ", base_year, " euro"),
  start_row = 4 + nrow(cmp_physical) + 3L,
  col_names = FALSE
)
wb <- wb_add_data(
  wb, "Comparison",
  x = cmp_constant,
  start_row = 4 + nrow(cmp_physical) + 4L
)

wb_save(
  wb,
  file.path(results_dir, "fish_provisioning_results.xlsx"),
  overwrite = TRUE
)
```

Joining results to the marine regions

Geometry lives in `data/gisdata/marine_regions.gpkg` and stays static. The published GeoPackage holds one copy of each polygon, with one column per year for landings, resource rent and price. Species, country and fleet belong in the RDS and Excel files, not in the geometry. Duplicating the polygons once per year was what made the first export large; three separate files, one per metric, would triplicate the geometry for no gain. QGIS users style `tonnes_2023`, `rent_const_k_eur_2018` or `price_const_eur_kg_2023` on the same layer. Nominal rent and price columns are kept beside the constant-price ones.

Two layers are published. The macro layer is the four pan-European sea regions; it joins on `Sea_name` after a label recode. The division layer is the 58 FAO divisions and subdivisions; it joins on `F_CODE`, or on `F_DIVISION` when that code is unique to one feature, which is how the collapsed NEAO recodes find the dissolved polygons. Price is the quantity-weighted average, revenue over live weight, not a sum of prices.

There is no legal-EEZ layer. `marine_regions.gpkg` is FAO fishing-area geometry only, and `fish_provisioning.gpkg` therefore has the same two result layers. Methods Tables 10 to 13 label FAO/GFCM `sub_reg_clean` as EEZ on the supply tables; that is the spatial unit of supply, not a coastal-state exclusive economic zone. Landing-country tables use STECF `country_code` (Eq. 2) and must not be drawn as if they were EEZs: the Member State of landing is not the coastal state of the fishing ground. Maps stay on the FAO layers.

Mediterranean accounts use GFCM sub-areas (`sa1` to `sa27`) while the geometry is FAO `37.x.x`, so the GSA lookup is applied before the join. Black Sea sub-areas `sa28` to `sa30` are not in that workbook; they are mapped to Marmara, the Black Sea and the Azov Sea following the GFCM numbering. ROW cannot appear on a European map. In 2023 it was 21 percent of tonnage, so any map needs that reconciliation in mind. Legacy result columns on the source layers (`Tonnes_23`, `Money_23` and the truncated overuse names) are not copied.

```r
black_sea_to_fao <- tribble(
  ~sub_reg_clean, ~fao_division,
  "sa28", "37.4.1",
  "sa29", "37.4.2",
  "sa30", "37.4.3"
)

sub_reg_to_map_code <- bind_rows(
  gsa_to_fao |>
    transmute(
      sub_reg_clean = str_remove_all(sub_reg, "\\s"),
      map_code = fao_division
    ),
  black_sea_to_fao |>
    transmute(sub_reg_clean, map_code = fao_division)
)

sea_reg_to_name <- tribble(
  ~Sea_reg,             ~Sea_name,
  "NEAO",               "NEAO",
  "Mediterranean Sea",  "MED",
  "Black Sea",          "Black",
  "Baltic Sea",         "Baltic"
)

european_seas <- sea_reg_to_name$Sea_reg
panel_years <- sort(unique(fish_provisioning$year))

wide_metrics <- function(data, id_cols) {
  widen <- function(value_col, prefix) {
    data |>
      select(all_of(id_cols), year, all_of(value_col)) |>
      pivot_wider(
        names_from = year,
        values_from = all_of(value_col),
        names_prefix = prefix
      )
  }
  widen("landed_tonnes", "tonnes_") |>
    left_join(widen("revenue_thousand_eur", "rent_k_eur_"), by = id_cols) |>
    left_join(widen("revenue_thousand_eur_constant", "rent_const_k_eur_"), by = id_cols) |>
    left_join(widen("price_eur_per_kg", "price_eur_kg_"), by = id_cols) |>
    left_join(widen("price_eur_per_kg_constant", "price_const_eur_kg_"), by = id_cols) |>
    select(
      all_of(id_cols),
      all_of(paste0("tonnes_", panel_years)),
      all_of(paste0("rent_k_eur_", panel_years)),
      all_of(paste0("rent_const_k_eur_", panel_years)),
      all_of(paste0("price_eur_kg_", panel_years)),
      all_of(paste0("price_const_eur_kg_", panel_years))
```

```
    )
}

by_macro <- fish_provisioning |>
  filter(Sea_reg %in% european_seas) |>
  summarise(
    landed_kg = sum(landed_kg),
    revenue_eur = sum(revenue_eur),
    revenue_eur_constant = sum(revenue_eur_constant),
    .by = c(year, Sea_reg)
  ) |>
  left_join(sea_reg_to_name, by = "Sea_reg") |>
  mutate(
    landed_tonnes = landed_kg / 1000,
    revenue_thousand_eur = revenue_eur / 1000,
    revenue_thousand_eur_constant = revenue_eur_constant / 1000,
    price_eur_per_kg = if_else(landed_kg > 0, revenue_eur / landed_kg, NA_real_),
    price_eur_per_kg_constant = if_else(
      landed_kg > 0, revenue_eur_constant / landed_kg, NA_real_
    )
  )

macro_geom <- st_read(gis_file, layer = "sea_regions_macro", quiet = TRUE) |>
  select(Sea_name)

macro_out <- macro_geom |>
  left_join(wide_metrics(by_macro, c("Sea_name", "Sea_reg")), by = "Sea_name")

division_geom <- st_read(gis_file, layer = "sea_regions_division", quiet = TRUE) |>
  select(F_CODE, F_DIVISION, NAME_EN)

division_join_keys <- bind_rows(
  st_drop_geometry(division_geom) |>
    transmute(map_code = F_CODE, F_CODE),
  st_drop_geometry(division_geom) |>
    summarise(n_features = n(), F_CODE = F_CODE[1], .by = F_DIVISION) |>
    filter(n_features == 1) |>
    transmute(map_code = F_DIVISION, F_CODE)
) |>
  distinct(map_code, .keep_all = TRUE)

by_division <- fish_provisioning |>
  filter(Sea_reg %in% european_seas) |>
  left_join(sub_reg_to_map_code, by = "sub_reg_clean") |>
  mutate(map_code = coalesce(map_code, sub_reg_clean)) |>
  inner_join(division_join_keys, by = "map_code") |>
  summarise(
    landed_kg = sum(landed_kg),
    revenue_eur = sum(revenue_eur),
    revenue_eur_constant = sum(revenue_eur_constant),
```

```
    .by = c(year, F_CODE)
  ) |>
  mutate(
    landed_tonnes = landed_kg / 1000,
    revenue_thousand_eur = revenue_eur / 1000,
    revenue_thousand_eur_constant = revenue_eur_constant / 1000,
    price_eur_per_kg = if_else(landed_kg > 0, revenue_eur / landed_kg, NA_real_),
    price_eur_per_kg_constant = if_else(
      landed_kg > 0, revenue_eur_constant / landed_kg, NA_real_
    )
  )

division_out <- division_geom |>
  left_join(wide_metrics(by_division, "F_CODE"), by = "F_CODE")

gpkg_out <- file.path(gis_dir, "fish_provisioning.gpkg")
if (file.exists(gpkg_out)) {
  file.remove(gpkg_out)
}
```

```
[1] TRUE
```

```
st_write(macro_out, gpkg_out, layer = "sea_regions_macro", quiet = TRUE)
st_write(division_out, gpkg_out, layer = "sea_regions_division", quiet = TRUE)
```

A short coverage check for 2023: how much of the European-sea tonnage found a polygon, and how much of the total was ROW and therefore off the map.

```
mapped_2023 <- by_division |>
  filter(year == reference_year) |>
  summarise(mapped_tonnes = sum(landed_tonnes))

european_2023 <- fish_provisioning |>
  filter(year == reference_year, Sea_reg %in% european_seas) |>
  summarise(european_tonnes = sum(landed_tonnes))

row_2023 <- fish_provisioning |>
  filter(year == reference_year) |>
  summarise(
    total_tonnes = sum(landed_tonnes),
    row_tonnes = sum(landed_tonnes[Sea_reg == "ROW"])
  )

tibble(
  european_tonnes = european_2023$european_tonnes,
  mapped_tonnes = mapped_2023$mapped_tonnes,
  unmapped_european_tonnes = european_2023$european_tonnes - mapped_2023$mapped_tonnes,
  row_tonnes = row_2023$row_tonnes,
  row_share_pct = 100 * row_2023$row_tonnes / row_2023$total_tonnes
```

european_tonnes	mapped_tonnes	unmapped_european_tonnes	row_tonnes	row_share_pct
2,681,188.2	2,554,121.6	127,066.6	713,110.3	21.0

```
) |>
  gt() |>
  fmt_number(
    c(european_tonnes, mapped_tonnes, unmapped_european_tonnes, row_tonnes),
    decimals = 1
  ) |>
  fmt_number(row_share_pct, decimals = 1)
```

Downloads

`_quarto.yml` lists `data/outputs/gis/` and `data/outputs/results/` under `project.resources`, so these files are copied into the published site. The links below are relative to this page.

```
tibble(
  File = c(
    "fish_provisioning.RDS",
    "fish_provisioning.xlsx",
    "fish_provisioning_results.xlsx",
    "rent_prices.RDS",
    "unvalued_records.RDS",
    "fish_provisioning.gpkg"
  ),
  Description = c(
    "Species-country-sub-region panel, 2002 to 2023, including 2010-euro rent",
    "The same panel as a single Excel sheet",
    "Report-style SUTs, one sheet per year: Europe Tables 2 to 9 and nested Tables 10 to 13 per se
    "Observed resource-rent prices used in gap filling",
    "Landings that could not be valued",
    "Macro and division polygons with tonnes, nominal rent and 2010-euro rent"
  ),
  Path = c(
    file.path("../../../data/outputs/results", "fish_provisioning.RDS"),
    file.path("../../../data/outputs/results", "fish_provisioning.xlsx"),
    file.path("../../../data/outputs/results", "fish_provisioning_results.xlsx"),
    file.path("../../../data/outputs/results", "rent_prices.RDS"),
    file.path("../../../data/outputs/results", "unvalued_records.RDS"),
    file.path("../../../data/outputs/gis", "fish_provisioning.gpkg")
  )
) |>
  mutate(
    Download = if (knitr::is_latex_output()) {
      sprintf("[%s](%s)", File, Path)
    } else {
      sprintf(
        '<a class="btn btn-primary btn-sm" href="%s" download>%s</a>',
```

File	Description
fish_provisioning.RDS	Species-country-sub-region panel, 2002 to 2023, including 2010-euro rent
fish_provisioning.xlsx	The same panel as a single Excel sheet
fish_provisioning_results.xlsx	Report-style SUTs, one sheet per year: Europe Tables 2 to 9 and nested Tables 10 to 13 per sea, in tonnes, nominal euro and 2010 euro
rent_prices.RDS	Observed resource-rent prices used in gap filling
unvalued_records.RDS	Landings that could not be valued
fish_provisioning.gpkg	Macro and division polygons with tonnes, nominal rent and 2010-euro rent

```
        Path, File
      )
    }
  ) |>
  select(Download, Description) |>
  gt() |>
  fmt_markdown(Download) |>
  cols_label(Download = "File")
```

Decisions from JRC (September 2026)

Domenico's answers to the open points, and what the routine now does.

- **Base year.** Constant prices use 2010 (`base_year` in the parameters chunk). Change that one line and re-render to switch; do not fold a new deflator into the allocation.
- **Subsidies on investments.** Operating subsidies (`totdirsub`) sit inside `variable_group == "Income"`, so the Income plus Expenditure filter already captures them. Subsidies on investments (`subinvest`) sit under `Capital` and remain excluded. Box 1 of the methods report will be corrected so that it no longer appears to include them.
- **`Type` and `Type_1`.** The published tables nest species group (fish / shellfish / other) with fish type (benthic, benthopelagic, demersal, pelagic). That is our `species_group` and `fish_type`. In the classification workbook, `Type` is the water-column typology; the fish / shellfish split is the binary flags. There is no `Type_1` column. The original-script names were the other way around relative to Domenico's note; the table columns are what we match.
- **Duplicate species codes.** The eight duplicated codes are identical pairs. They are dropped with `distinct()`.
- **Mediterranean sub-areas `sa11.1` and `sa11.2`.** Recoded to `sa11` and counted in the Mediterranean. The 2023 shapefile tonne anchors for Mediterranean and ROW therefore shift by that tonnage.
- **EEZ.** STECF has no coastal-state EEZ field. Supply tables label `sub_reg_clean` as EEZ, with a note that these are FAO/GFCM fishing areas. Maps stay on FAO layers. Use tables are landing country.
- **Overuse.** Wired after allocation from the 2018 FSI scores. It is a companion overlay, not a column of the official SUT. Scores are held constant across years until a new stock-assessment vintage arrives.
- **2002 to 2007 (October 2026).** Added from the 2011 DCF data call, with fleets defined as country plus vessel length and operating cash flow as the net result, following Domenico's description of the dataset. See Extending the series back to 2002.

References

Eurostat. (2026). *HICP annual data (average index and rate of change) (prc_hicp_aind)*. Retrieved from https://ec.europa.eu/eurostat/databrowser/view/prc_hicp_aind

Froese, R., & Pauly, D. (2024). *FishBase*. World Wide Web electronic publication. Retrieved from https://www.fishbase.org

Haines-Young, R., & Potschin, M. B. (2018). *Common international classification of ecosystem services (CICES) V5.1 and guidance on the application of the revised structure*. Fabis Consulting Ltd. Retrieved from Fabis Consulting Ltd. website: https://cices.eu

Palomares, M. L. D., & Pauly, D. (2024). *SeaLifeBase*. World Wide Web electronic publication. Retrieved from https://www.sealifebase.org

Scientific, Technical and Economic Committee for Fisheries. (2024). *The 2024 annual economic report on the EU fishing fleet (STECF 24-07)*. Luxembourg: Publications Office of the European Union. Retrieved from Publications Office of the European Union website: https://stecf.ec.europa.eu/data-dissemination_en

United Nations, European Union, Food and Agriculture Organization of the United Nations, International Monetary Fund, Organisation for Economic Co-operation and Development, & World Bank. (2014). *System of environmental-economic accounting 2012: Central framework*. New York: United Nations. Retrieved from https://seea.un.org/sites/seea.un.org/files/seea_cf_final_en.pdf